

PROGRAMAÇÃO WEB

AULAS TEÓRICAS

ANO LECTIVO DE 2024/25

1° SEMESTRE



Capítulo 2

Construção de Websites em .NET Core 8

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

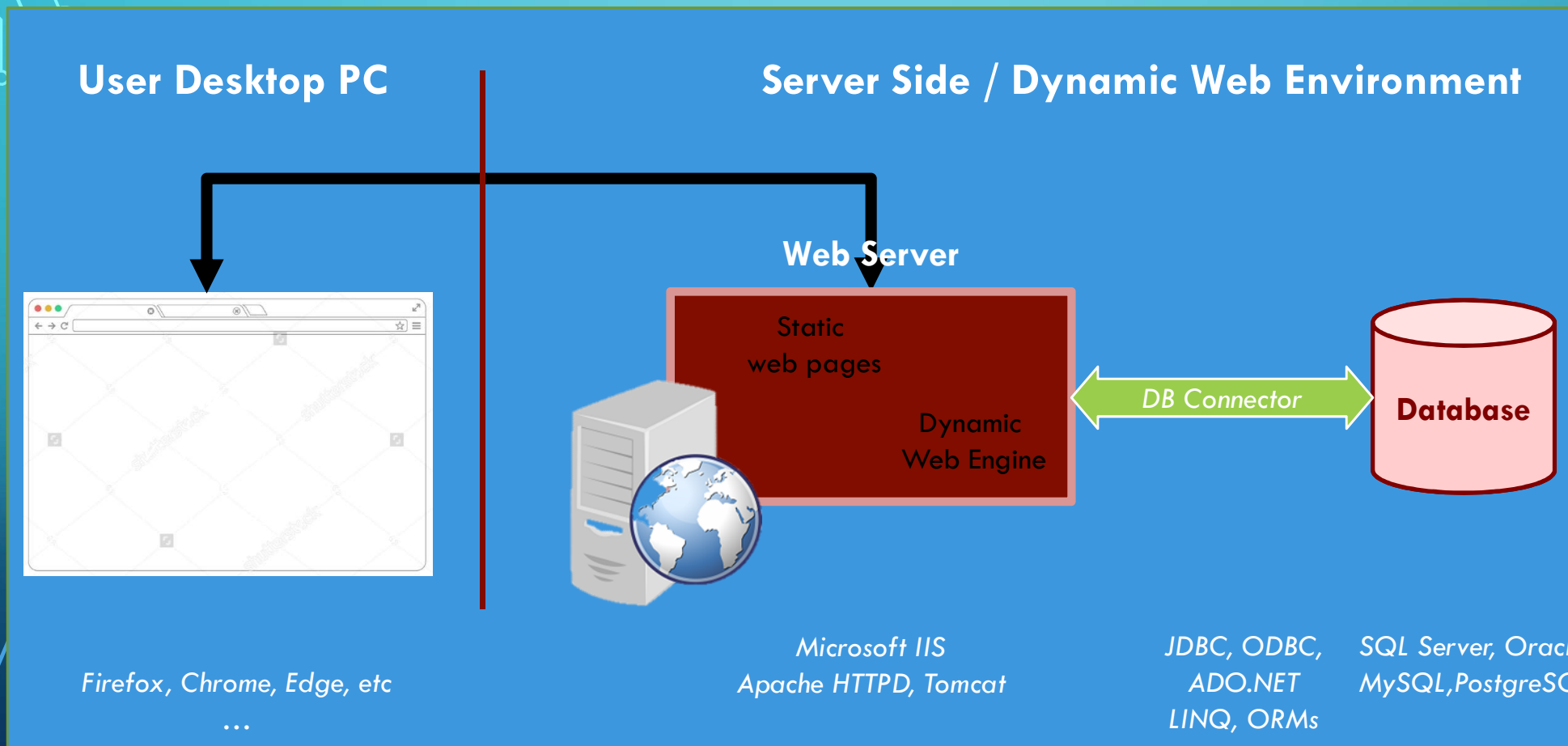
2.1. Introdução ao .*Net* Core 8

Programação Web – 2024/2025 – LEI D+C2+PL – ISEC/IPC - JB

APLICAÇÕES WEB

- Conteúdo Estático
  - Não pode ser atualizado ou alterado sem efetuar alterações no código-fonte – praticamente já não existem
- Conteúdo Dinâmico
  - Conteúdos podem ser alterados ou atualizados sem necessidade de efetuar alterações no código-fonte
  - Existem Duas Perspectivas da noção de Dinamismo: alteração de características dos elementos HTML e alteração do conteúdo das páginas

ARQUITECTURA CLÁSSICA TÍPICA



TECNOLOGIAS E *FRAMEWORKS*

C# 9

LINQ
Language Integrated Query

ASP.NET Core

ASP.Net Core Entity Framework

ASP.Net Core Identity

Razor
(Razor View Engine)

E HTML?
E Java Script?
E CSS?



JSP

PHP

Ruby

Node.js

• • •

.Net Core 9
Blazor

Programação Web – 202

ASP.NET 8 CORE



- Framework da Microsoft para desenvolvimento de aplicações Web.
- A actual versão do ASP.Net Core, permite desenvolver aplicativos da Web baseadas na Cloud usando MVC, Razor Pages e Blazor:
  - *Server-side + Client Side*
  - *Blazor Server*
  - *Blazor WebAssembly (WAsm)*
  - *Blazor MAUI*

Programação Web – 2024/2025 – TLD+CSS+PL – ISEC/IPC - JM

ASP.NET 8 CORE

- Este framework está directamente relacionado com o .NET Framework
- Sucessor da tecnologia ASP e ASP.Net MVC
- Também é Baseado em eventos
- Apesar de se poder continuar a usar uma diversidade de linguagens de programação, C#, F#, VB.NET (parcialmente), no desenvolvimento em .Net tem sido usado preferencialmente o C#, nomeadamente a última versão 12.0

Programação Web – 2024/2025 – LEI DTSF+PL – ISEC/IPC – JB

EVOLUÇÃO DO ASP.NET -> .NET -> .NET CORE



Active Server Pages (Classic ASP)

1996

ASP.NET (Web Forms)

2002

2008

ASP.NET MVC 1/2/3/4/5

2010

ASP.NET Web Pages

ASP.NET Web API
ASP.NET SignalR

2012

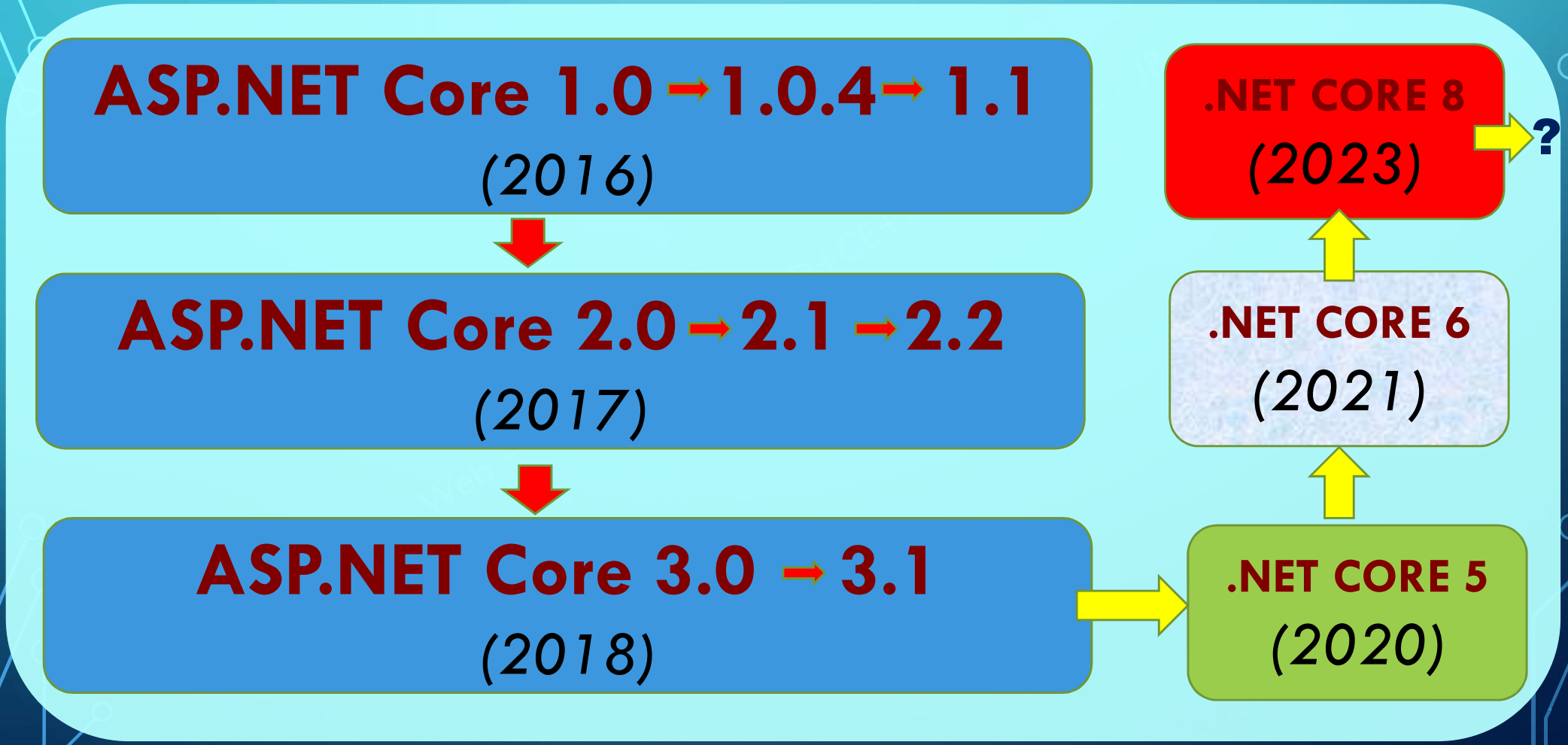
Desde 2015

ASP.NET ~~MVC 6~~
Core ~~MVC~~
.NET Core

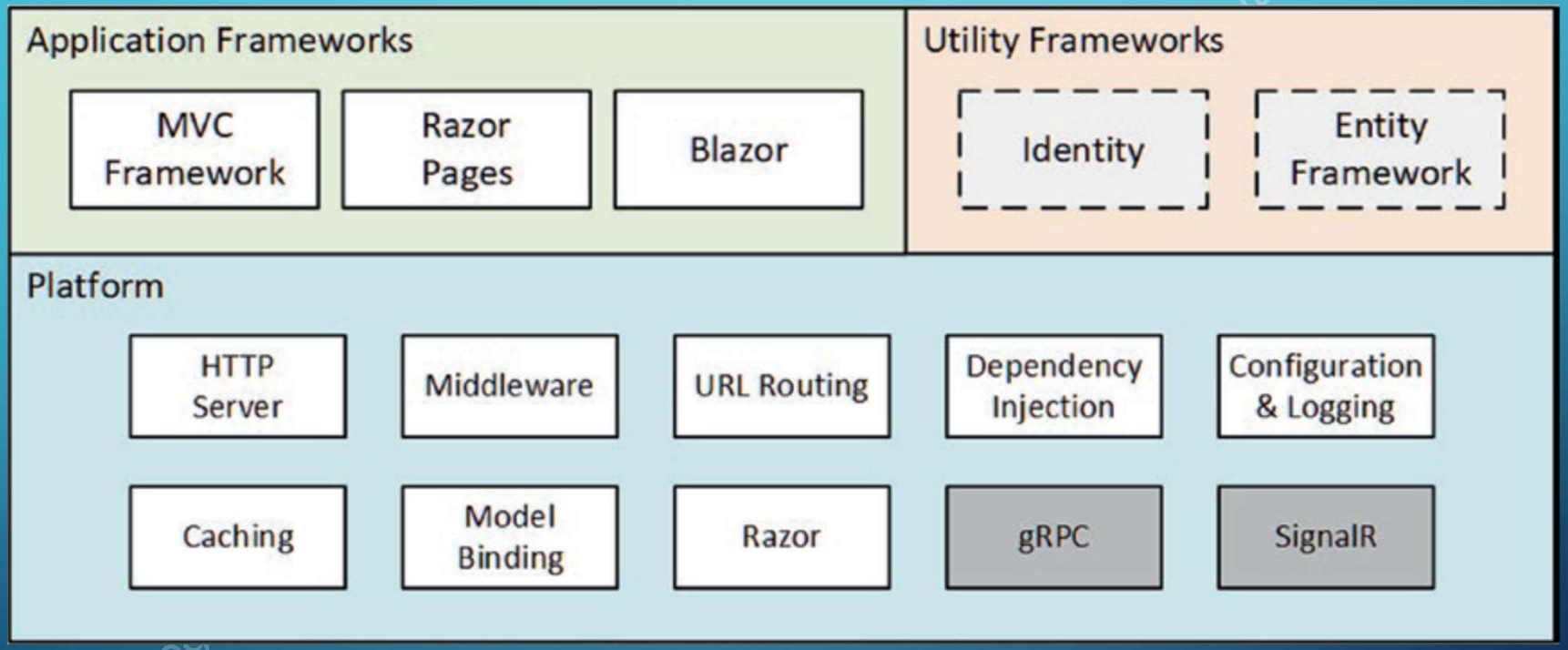
.Net Core 8

Razor | Blazor
(Browser+ Razor)
(Server | Wasm)
(MAUI)

EVOLUÇÃO DO .NET CORE



ASP.NET CORE



Estrutura do ASP.Net Core [1]

ASP.NET CORE



- O ASP.NET original dependia de um modelo de desenvolvimento designado por Web Pages, que siimplificava o desenvolvimento de sites mas os projectos utilizados para o seu desenvolvimento não eram fáceis de gerir
- O MVC Framework, a versão anterior ao actual ASP.Net Core foi posteriormente introduzido e era utilizado em conjunto com as Web Pages com um modelo de desenvolvimento
  - MVC significa Model-View-Controller, é um padrão de design através do qual as áreas de funcionalidade, também designadas por separação de interesses, são definidas de forma independente

Programação Web – 2024/2025 – LEI D+CE+RL – ISEC/IPC – JB

ASP.NET CORE



Sites | **Serviços**

Web Forms	Web Pages	Single Page Apps	MVC	Core	.Net	Web API	SignalR

ASP → **ASP.Net** → **.Net** → **.Net Core**

ASP.NET CORE

- ***Blazor***

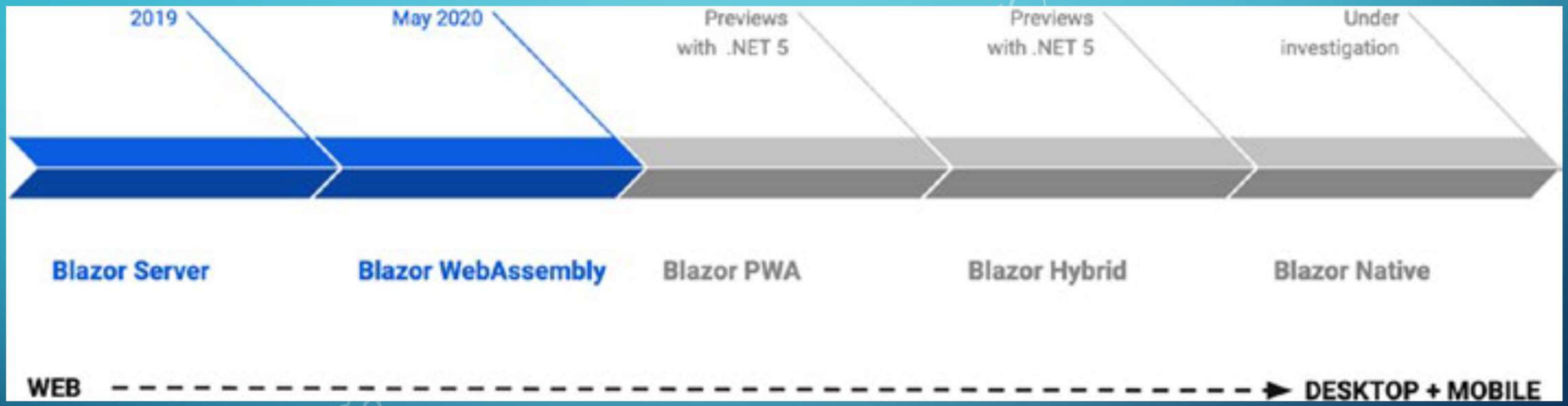


- O ***Blazor*** foi criado em 2017 como um projeto pessoal de Steve Sanderson, um Software engineer da Microsoft, que apresentou uma versão prévia do *Blazor* baseado em *DotNetAnywhere*, um *.NET* Intérprete de linguagem intermédia (IL)
  - Nesta altura o *Blazor* foi também adicionado ao repositório *ASP.Net GitHub* como um projeto experimental, e a grande adesão da comunidade convenceu a *Microsoft* a mover o projeto para a equipe *ASP.Net*, substituindo *DotNetAnywhere* com *Mono*, que era o código aberto mais conhecido da plataforma baseada no *.NET Framework*

Programação Web – 2024/2025 – LEI-D+CT+PL – DEC/I

ASP.NET CORE

- ***Road Map do Blazor***



Com o lançamento do ***.NET Core 3***, o ***Blazor*** tornou-se parte deste *framework* com um *road map* de desenvolvimento muito ambicioso como mostrado

ASP.NET CORE

Blazor

- Actualmente há várias variantes do Blazor:
  - O Blazor Server
  - O Blazor WebAssembly
  - O Blazor MAUI
- *A designação Blazor resulta da junção das designações Browser + Razor*
- O Blazor muda muito o paradigma atual da Programação Web, em particular o Blazor Web Assembly, Blazor WAsm
  - *A Microsoft lançou a versão RTM (release-to-manufacturing) do Blazor com o .NET Core 3, sendo uma nova estrutura de front-end que pretendia resolver alguns problemas*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

ASP.NET CORE

- ***Blazor***
  - Com o ***Blazor***, pode-se usar C# e *.NET* como estrutura central para escrever o *front-end* das aplicações,
  - ***Blazor*** permite também que se execute o *front-end* diretamente no browser, fornecendo todas as ferramentas necessárias para o desenvolvimento de aplicativos de página única, Blazor WASM
  - Blazor MAUI, permite o desenvolvimento de aplicações em código nativo para *devices* com plataformas e SO diferentes como o Windows, o Android, o iOS, o macOS e o Tizen
  - A conjugação num só projecto do Blazor Web e de Blazor MAUI permite o desenvolvimento com um único código, quer em C# quer em CSS, para a Web, "sites", quer para os *devices* em Windows, Android, iOS, macOS e Tizen recorrendo-se para isso à RCL, Razor Class Library

Programação Web – 2024/2025 – LEI DA CE+PO – ISEC/IPC – JB

ASP.NET CORE

- O ***Blazor WebAssembly*, WAsm,** executa o código C# compilado diretamente no navegador.
- O ***Blazor WebAssembly*** foi inicialmente distribuído com o ***.NET Core 3.1.300***
- O ***Blazor WAsm*,** traduz uma grande revolução no paradigma da Programação Web ao executar directamente no lado do cliente código que até aqui era somente executado do lado do servidor, como é o caso do **C#** *depois de* compilado

Programação Web – 2024/2025 – LEI D+CE+RAI – ISEC/IPC - JB

ASP.NET CORE



- O ***Blazor Hybrid*** é um renderizador ***.NET*** nativo *WebView*, que permite o desenvolvimento de aplicações nativas funcionando para o Windows, Android, iOS, macOS e Tizen

- ***Blazor Native*** de acordo com o previsto, terá o mesmo modelo de programação, mas sem renderização de *HTML*.

Programação Web – 2024/2025 – LEI D+CC+RL – ISEC/IPC

ASP.NET *WEB API*



- Framework que permite desenvolver serviços HTTP, API
  - REST-based *webservices*
- ***REST*** significa *Representational State Transfer*. Em português, **Transferência de Estado Representacional**. Trata-se de uma abstração da arquitectura da Web.
  - Resumidamente, o REST consiste em princípios/regras/constraints que, quando seguidas, permitem a criação de um projecto com interfaces bem definidas. Desta forma, permite, por exemplo, que aplicações comuniquem entre si

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



2.2. Novidades introduzidas com o C# 12 e com o .NET Core 8

O lançamento do . NET 8 e C# em novembro de 2023 foi um marco significativo no mundo . NET, introduzindo funcionalidades avançadas e melhorias que revolucionam o desenvolvimento de projetos ASP.NET Core.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

2.2. Novidades introduzidas com o C# 12 e com o .NET Core 8

- **Construtores primários**
  - A utilização de construtores primários pode tornar a declaração de construtores de classes mais simples e limpa.

- **Expressões de coleção**
  - Podemos utilizar as expressões de coleção como forma alternativa de inicializar as coleções.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8



- **Parâmetros Lambda padrão**
  - As expressões lambda permitem a criação de funções dentro de um método para realizar uma tarefa simples.

  Os parâmetros lambda padrão permitem-nos adicionar o valor padrão para o parâmetro na expressão lambda.
- **Alias de qualquer tipo**
  - Podemos criar um alias para um tipo de coleção para tornar as declarações mais simples e limpas

Programação Web – 2024/2025 – LEI-D+CE+RC – ISEC/DEC - JB

.NET CORE 8



- **Atributo Experimental**
  - Podemos marcar os recursos experimentais ou arriscados com um atributo que serve de marcador. Ao aplicar este atributo a uma funcionalidade ou bloco de código específico, indicamos que a funcionalidade associada está sujeita a avaliação e poderá sofrer alterações ou ser removida em futuras atualizações.
- **Inline Arrays**
  - Este recurso é de utilização muito rara e dificilmente será utilizado. No entanto, as matrizes inline podem melhorar o desempenho em alguns cenários específicos.

Programação Web – 2024/2025 – LEI D+CE-PL – ISEC/IPC - JP

2.3. Blazor e Modelos de Hospedagem Blazor no .Net Core 8



Programação We... – LEI D+C+PL – ISEC/DEI-PC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

- No Blazor Server, como o nome sugere, a lógica da aplicação é executada no servidor.
- O servidor Blazor utiliza uma ligação SignalR para fornecer as atualizações da interface do utilizador ao utilizador no seu browser a partir do servidor.
  - Aqui, quando o utilizador tenta aceder à sua aplicação web, o utilizador obterá a UI da sua aplicação enquanto a lógica de back-end permanece no servidor e é procurada como e quando solicitada.

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/DEC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

- Com a utilização do SignalR, podem ser vistas atualizações em tempo real entre o cliente e o utilizador, o que ajuda a manter a UI sincronizada com o estado do servidor.
  - Como o código é executado no servidor, tem-se acesso total às APIs .NET, bem como aos recursos do lado do servidor, como a base de dados, o sistema de ficheiros, etc.

Programação Web - 2024/2025 – LEI-CE+TP – ISEC/IPC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

Como funciona o servidor Blazor?

Suponha que acede a uma aplicação web desenvolvida no Blazor Web Server. Vejamos qual o processo que ocorrerá quando visitar este site pela primeira vez.

- Quando navega inicialmente para uma aplicação Blazor Web Server, o servidor enviará uma página HTML inicial que incluirá um pequeno código Javascript para estabelecer a ligação SignalR. Normalmente, o nome do javascript será blazor.server.js

Programação Web – 2024/2025 – LEI+CET+TeSP – ISEC/IPC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

- Este SignalR ajuda a estabelecer um canal de comunicação em tempo real entre cliente e servidor.
- A interação do utilizador, como cliques em botões ou envios de formulários, é enviada para o servidor através da ligação SignalR.
- O servidor processa então estes eventos utilizando a lógica da aplicação e envia as atualizações da UI de volta para o cliente através da ligação SignalR, e o browser atualiza o DOM em conformidade.

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/PC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

Benefícios do Servidor Blazor

- Como é necessária apenas uma pequena quantidade de dados, o tempo de carregamento é rápido.
- Isto é mais seguro, pois a lógica da aplicação não é exposta ao cliente.
- Todos os ecossistemas . NET podem ser aproveitados, incluindo APIs e bibliotecas do lado do servidor.

Programação Web 2024/2025 – LD+CE+RL – ISEC/IPC - JB

.NET CORE 8

2.3.1 *.Net Core 8 Blazor Server*

Desvantagens do servidor Blazor

- A latência da rede pode ocorrer se o servidor estiver lento.
- Como o servidor tem de lidar com múltiplas ligações SignalR, as aplicações de alto tráfego podem enfrentar problemas.
- A aplicação não pode funcionar se não estiver ligada ao servidor.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

2.3.2 *.Net Core 8 Blazor WebAssembly*

O Blazor WebAssembly é executado inteiramente no browser e utiliza o WebAssembly (também conhecido como Wasm) para executar o código .NET. O Wasm permite-nos implantar na web para aplicações cliente e servidor. O Blazor WebAssembly utiliza a compilação Ahead of Time (AOT), onde podemos compilar o nosso código .NET diretamente no WebAssembly. Sem utilizar AOT, o Blazor WebAssembly pode ser executado no browser utilizando o interpretador .NET Intermediate Language implementado no WebAssembly com suporte parcial de compilação just-in-time (JIT).

Programação Web - 2024/2025 - LEI D - CE+PL - ISEC/APC - JB

.NET CORE 8

2.3.2 *.Net Core 8 Blazor WebAssembly*

O Blazor WebAssembly é executado inteiramente no browser e utiliza o WebAssembly (também conhecido como Wasm) para executar o código .NET. O Wasm permite-nos implantar na web para aplicações cliente e servidor. O Blazor WebAssembly utiliza a compilação Ahead of Time (AOT), onde podemos compilar o nosso código .NET diretamente no WebAssembly. Sem utilizar AOT, o Blazor WebAssembly pode ser executado no browser utilizando o interpretador .NET Intermediate Language implementado no WebAssembly com suporte parcial de compilação just-in-time (JIT).

Programação Web – 2024/2025 – LEI D+CF+PL – ISEC/AnC – JB

.NET CORE 8

2.3.2 *.Net Core 8 Blazor WebAssembly*

Como funciona o Blazor WebAssembly?

- Quando navega para uma aplicação WebAssembly, o servidor envia os ficheiros estáticos, incluindo o tempo de execução do WebAssembly, os assemblies .NET e quaisquer outros ativos necessários para a aplicação.
- O tempo de execução do WebAssembly no browser carrega e executa os assemblies .NET.
- A estrutura Blazor trata da renderização da UI e da gestão das interações do utilizador.
- A aplicação Blazor do lado do cliente pode comunicar com as APIs do lado do servidor utilizando chamadas HTTP padrão.

Programação Web – 2024/2025 – LEI-D+CET-PL – ISEC/IPC - JB

.NET CORE 8

2.3.2 *.Net Core 8 Blazor WebAssembly*

Benefícios do Blazor WebAssembly

- Fornece suporte offline após o carregamento inicial enquanto a aplicação é executada no lado do cliente.
- Aproveita o poderoso ecossistema . NET, permitindo a reutilização de código e uma experiência de desenvolvimento unificada.
- Funciona em qualquer browser que suporte WebAssembly sem necessidade de plugin.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



2.3.2 *.Net Core 8 Blazor WebAssembly*

Desvantagens do Blazor WebAssembly

- O tempo de download inicial é bastante grande, uma vez que todos os assemblies e o tempo de execução do .NET têm de ser descarregados para o browser.
- As APIs do lado do servidor não são acessíveis diretamente, portanto restritas às APIs disponíveis no WebAssembly e no browser.

Programação Web – 2024/2025 – LEI D+GI+RL – ISEC – APC - JB

.NET CORE 8

2.3.3 .Net Core 8 Blazor Hybrid

O Blazor Web App oferece uma abordagem híbrida, suportando a interatividade do lado do servidor e do cliente. Combina os benefícios do Blazor WebAssembly e também do Blazor Server. Isto permite-nos construir aplicações que podem ser renderizadas tanto no lado do cliente usando WebAssembly como no lado do servidor, dependendo do cenário. Pode oferecer melhor desempenho, escalabilidade e flexibilidade aproveitando os pontos fortes de ambas as abordagens.

Programação Web – 2024/2025 – LEI-D+CE-PL – ISEC/IPC - JB



2.3.3 .Net Core 8 Blazor Hybrid

Como funciona o Blazor Web App?

- Suponha que navega até ao URL da aplicação que utiliza o Blazor Web App Hosting Model.
- A página inicial pré-renderizada é servida pelo servidor e é visível para o utilizador enquanto o browser descarrega o tempo de execução do Blazor WebAssembly em segundo plano.

Programação Web – 2024/2025 – LEI D+CE+RL – ISEC/IPC - JB

2.3.3 .Net Core 8 Blazor Hybrid

Como funciona o Blazor Web App?

- A renderização WebAssembly inicializa e assume o controlo da renderização, fazendo a transição da aplicação para execução no lado do cliente.
- O cliente trata das chamadas API subsequentes e das atualizações da UI, para que a necessidade de comunicação com o servidor possa ser reduzida.

Programação Web – 2024/2025 – UFCD 0784 – PL – ISEC/IPC - JB

.NET CORE 8

2.3.3 .Net Core 8 Blazor Hybrid

Benefícios da aplicação Web Blazor

- Carregamento inicial rápido, uma vez que o carregamento inicial é pré-renderizado pelo servidor.
- A pré-renderização no servidor garante que os motores de pesquisa podem indexar o conteúdo da aplicação, o que ajuda no SEO.
- Depois de o WebAssembly ser inicializado, a aplicação poderá executar a interação de manipulação no browser, o que reduz a interação do servidor e aumenta a capacidade de resposta.

Programação Web – 2024/2025 – LEI D-PGE+PL – ISEC/IPC - JB

2.3.3 .Net Core 8 Blazor Hybrid

Benefícios da aplicação Web Blazor

- O servidor não é obrigado a processar todas as interações que libertam recursos do servidor para outras tarefas.

Programação Web – 2024/2025 – TeSP D+GT+PL – ISEC/IPC - JB

.NET CORE 8

2.3.3 .Net Core 8 Blazor Hybrid

Desvantagens da aplicação Web Blazor

- A complexidade aumenta à medida que precisamos de gerir eficientemente tanto o lado do servidor como o lado do cliente.
- O tamanho do download é grande, uma vez que o tempo de execução do . NET e os assemblies são descarregados para o browser.
- A segurança necessita de ser tratada tanto para aplicações do lado do cliente como do lado do servidor.

Programação Web – 2024/2025 – LEI D+GI+PL – ISEC/IPC - JB

.NET CORE 8

Qual o modelo de alojamento que deve escolher?

A escolha entre o Blazor Server, o Blazor WebAssembly e o Blazor Web App depende de vários fatores:

- Requisitos da aplicação,
- Necessidades do utilizador final
- Bapacidades de desenvolvimento

Programação Web – 2024/2025 – LEI D+CB PL – ISEC/IPC - ...

.NET CORE 8



Alguns dos critérios que o podem ajudar a tomar esta decisão são:

1. Servidor Blazor

- Blazor Server é melhor para aplicações que têm lógica pesada no lado do servidor.
- Pode ser utilizado em cenários onde o SEO e o tempo de carregamento inicial são críticos.
- Pode ser utilizado em cenários onde teremos uma conectividade de rede consistente (como aplicações empresariais internas).

Programação Web 2024/2025 – LEI+LD+CE+PL – ISEC/IPC - JB

.NET CORE 8

2. Blazor WebAssembly

- Os projetos Blazor WebAssembly são melhores para aplicações orientadas para o cliente, onde nos concentramos na experiência do utilizador e na capacidade de resposta.

- Pode ser utilizado em aplicações que requerem conectividade offline.

Programação Web – 2024/2025 – LEI D+2L+PL – ISEC/IPC - JB

.NET CORE 8

3. Aplicação Web Blazor

- Pode ser utilizado em aplicações que exigem equilíbrio entre desempenho, escalabilidade e experiência do utilizador.
- Pode ser utilizado em cenários onde o SEO e o tempo de carregamento inicial são críticos, mas também são necessários capacidade de resposta e suporte offline.

Tendo esta matriz em mente, pode tomar uma decisão relacionada com qual o modelo de alojamento que seria mais adequado às suas necessidades.

Programação Web – 2024/2025 – TeSP D+CE – ISEC/IPC - JB



2.3.4 .Net Core 8 MAUI

.NET MAUI é uma estrutura de UI multiplataforma para construir aplicações nativas para iOS, Android e Windows utilizando uma única base de código.

- É uma evolução do Xamarin.Forms, o qual foi descontinuado

O .Net MAUI inclui novas funcionalidades e melhorias que abordam algumas das limitações do Xamarin.Forms.

Programação Web – 2024/2025 – LEI-D+CE+PL – ISEC/IPC - JB

.NET CORE 8

2.3.4 .Net Core 8 MAUI

Uma das principais vantagens do . NET MAUI é que suporta mais plataformas do que o Xamarin.Forms, incluindo macOS, Tizen e watchOS.

Possui também uma arquitetura mais flexível que permite aos programadores criar controlos e layouts personalizados com mais facilidade.

O . NET MAUI utiliza o . NET a partir da versão 6 e foi substancialmente melhorado na versão 8, que inclui melhorias de desempenho e novas funcionalidades de linguagem.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

Arquitetura

O .NET MAUI possui uma arquitetura mais flexível e extensível

- *Utiliza um novo standard MVU (Model-View-Update), que permite uma maior dissociação entre a UI e a lógica da aplicação. Isto facilita a criação de controlos e layouts personalizados e o teste da lógica de negócio da aplicação independentemente da IU.*

Programação Web – 2024/2025 – LEI D+ PL – ISEC/IPC - JB

.NET CORE 8

Suporte de plataforma

O .NET MAUI suporta mais plataformas que o Xamarin.Forms.

- *Inclui suporte para macOS, Tizen e watchOS, bem como iOS, Android e Windows.*
- *Isto torna mais fácil para os programadores criarem aplicações direcionadas a um público mais amplo.*

Programação Web - 2024/2025 – LEI D+CE+PL - ISEC/IPC - JB

.NET CORE 8

Desempenho

O . NET MAUI inclui subatanciais melhorias de desempenho

Utiliza o .NET a partir da versão 6, que inclui melhorias no tempo de execução e no garbage collector.

- *Como resultado temos tempos de arranque mais rápidos, menor utilização de memória e animações mais suaves.*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

Ferramentas

- *O . NET MAUI inclui ferramentas melhoradas*

- *Inclui um novo sistema de projetos que simplifica o processo de criação e gestão de projetos.*

- *Inclui também um novo editor visual para criar layouts de UI.*

Programação Web – 2024/2025 – LEI D+CE+PL – MC/IPC - JB

.NET CORE 8

2.3.5 .Net Core 8 Razor Class Library

- *As Razor Class Librairies são um recurso muito interessante que apareceu com o ASP.Net Core 2.1 mas que tem vindo a ser melhorado e como tal bastante uitilizado actualmente*
- Este novo tipo de recurso permite encapsular em uma biblioteca construções como **Componentes Razor, Razor Pages, Views** e **Controllers,** contribuindo assim para um maior reaproveitamento de código em projetos Web baseados no **ASP.NET Core**

Programação Web – 2024/2025 – LEI-D+CE+RL – ISEC/IPC - JB

.NET CORE 8

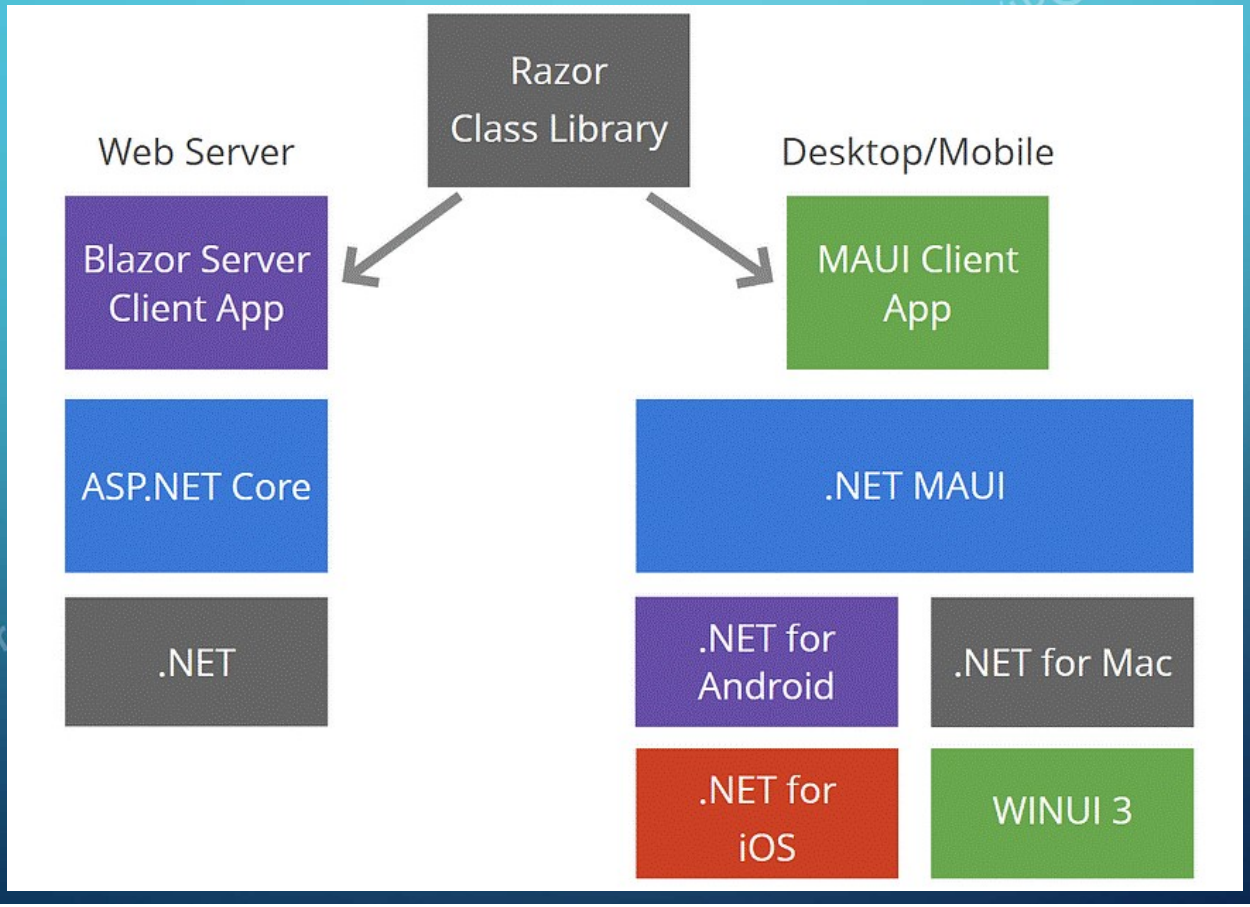


- *No .Net Core 8, a utilização e conjugação das Razor Class Librairies com o .Net 8 Blazor Web e o .Net 8 Blazor MAUI, permite construir aplicações hibridas que conjugando estes três tipos de recursos nos permitem com um único código construir aplicações que corram num browser, ou em código nativo em dispositivos como Smartphones ou Tablets, iOS e Android, em Windows, em macOS ou em Tizen*

Programação Web – 2024/2025 – LEI DTPE+PL – ISEC/IPC

.NET CORE 8

Desenvolvimento Hibrido com Blazor Web + Blazor MAUI + Blazor RCL



.NET CORE 8

Desenvolvimento Hibrído com Blazor Web, Blazor MAUI e Blazor RCL



- **Base de código única:** os programadores podem escrever um único código para a lógica de UI e lógica de negócio que é executado em diversas plataformas, reduzindo o tempo e o esforço de desenvolvimento.

- **Controlos nativos:** O MAUI fornece acesso a controlos e recursos nativos, garantindo que as aplicações tenham a aparência de uma aplicação nativa em cada plataforma.

Programação Web – 2024/2025 – LEI+CE+LEC – ISEC/IPC - JB

.NET CORE 8

Desenvolvimento Hibrido com Blazor Web, Blazor MAUI e Blazor RCL



- **Personalização específica do dispositivo:** os programadores podem adaptar a UI/UX a diferentes dispositivos e plataformas, otimizando a experiência do utilizador sem sacrificar o desempenho nativo.

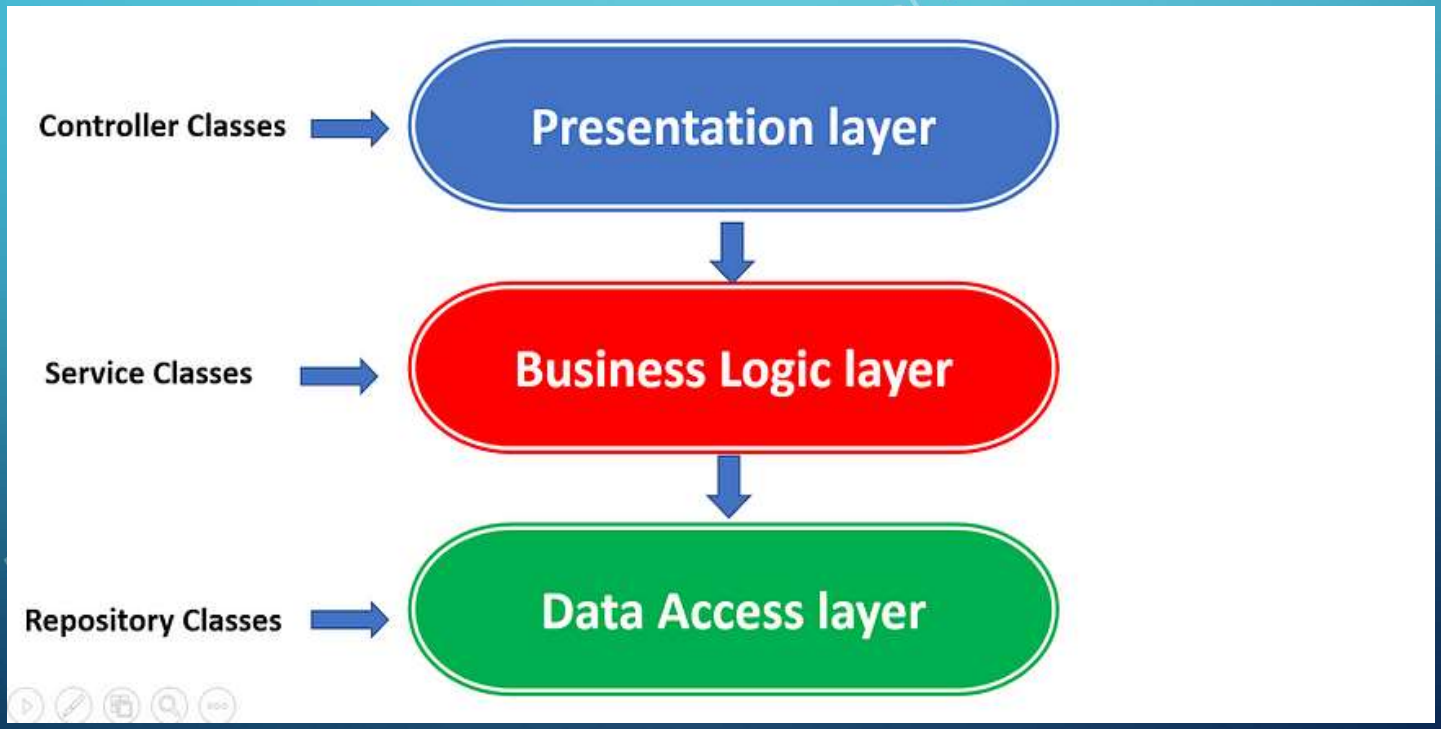
- **UI adaptável:** O MAUI suporta o design de UI adaptável, permitindo que as aplicações adaptem o seu layout e comportamento com base em fatores como o tamanho do ecrã, orientação e diretrizes específicas da plataforma.

Programação Web – 2024/2025 – LEI D+G+PL – ISEC/IPC - JB

.NET CORE 8

2.6. Arquitecturas para desenvolvimento Web

1. Tree N-Tier Architecture



.NET CORE 8

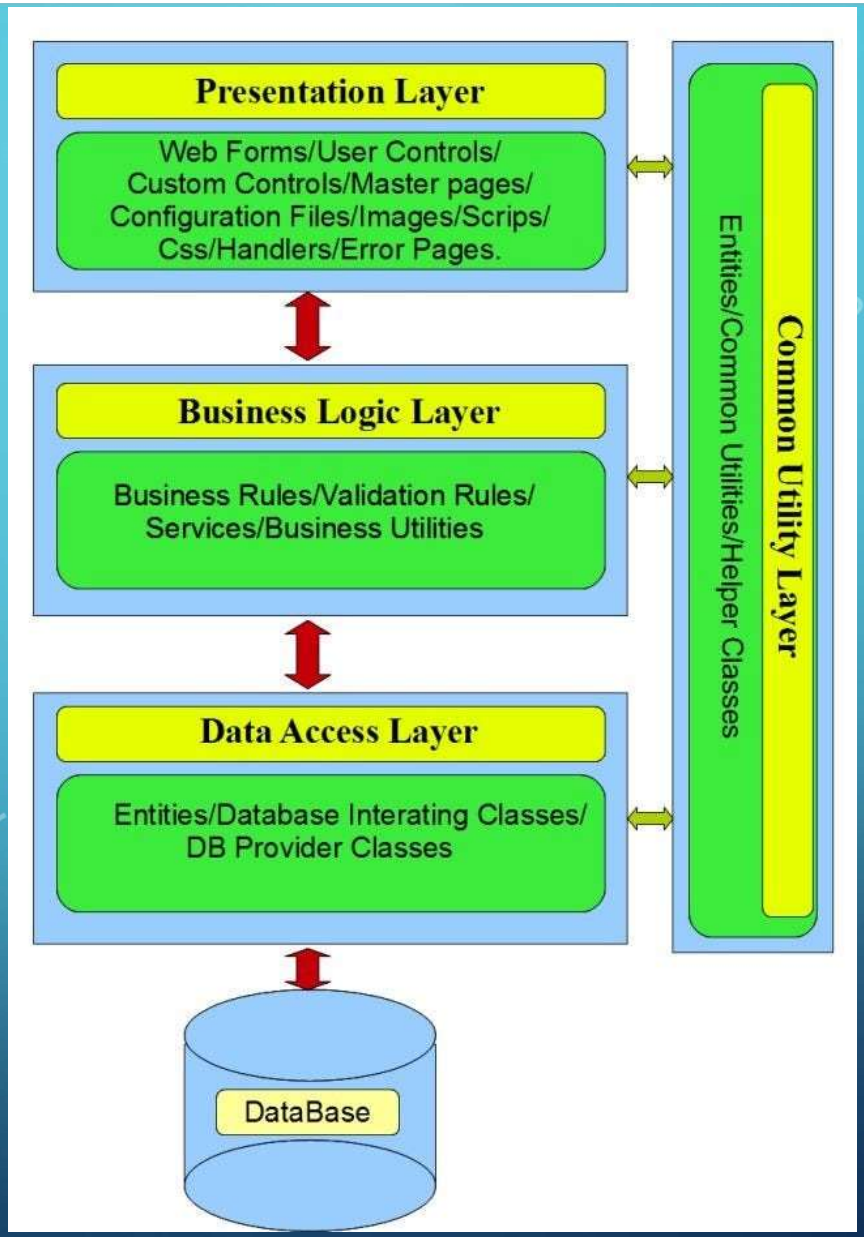
A arquitetura em camadas no . NET é utilizada para dividir aplicações complexas em partes ou camadas mais pequenas e mais facilmente gerenciáveis.

Cada camada concentra-se num aspeto específico, como a apresentação, a lógica de negócio ou o acesso a dados.

- *Esta abordagem oferece benefícios como modularidade, separação de interesses, escalabilidade, flexibilidade, capacidade de reutilização e melhor testabilidade.*
- *É muitas vezes implementada utilizando tecnologias como ASP.NET MVC, .NET Core, Entity Framework, etc.*

Programação Web – 2024/2025 – LEI D-TSE+PL – ISEC/IPC – JB

.NET CORE 8



Programação Web –

1.ª Camada de apresentação

- Esta camada é responsável por apresentar informação ao utilizador e processar inputs do utilizador.
  - Inclui componentes como interfaces de utilizador, páginas web ou terminais API.
  - O seu objetivo é interagir com os utilizadores e traduzir as suas ações em pedidos que a aplicação possa "compreender"

Programação Web – 2024/2025 – LEI DTSE+PL – ISEC/IPC - JB

2.º Camada Lógica de Negócio (Camada de Serviço)

- A camada de lógica de negócio contém as principais funcionalidades e regras da aplicação.
  - Implementa processos de negócio, fluxos de trabalho, validações e cálculos.
  - Tem como finalidade impor regras de negócio, gerir a integridade dos dados e orquestrar interações entre diferentes partes do sistema.

Programação Web – 2024/2025 – LEI D+ORT+PL – ISEC/IPC - JB

.NET CORE 8

3.ª Camada de Acesso a Dados

- Esta camada trata das interações com o armazenamento de dados subjacente, como uma base de dados ou serviços externos.
  - Inclui componentes responsáveis por consultar, inserir, atualizar e eliminar dados.
  - O seu objetivo é abstrair os detalhes da implementação do armazenamento de dados e fornecer uma interface consistente para aceder e manipular os dados.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

2. Clean Architecture

O termo "Arquitetura Limpa" tem vindo a tornar-se cada vez mais popular no desenvolvimento de software nos últimos anos.

- *Clean Architecture é um padrão de design de software que prioriza a separação de interesses, tornando mais fácil manter, testar e evoluir uma aplicação ao longo do tempo. Iremos analisar em particular a utilização da Clean Architecture em aplicações ASP.NET Core.*

Programação Web – 2024/2025 – LEI D+CE+PL, ISEC/DPC - JB

O que é a Arquitetura Limpa?

A Arquitetura Limpa é um padrão de design que separa uma aplicação em diferentes camadas com base na sua responsabilidade.

- *É uma forma de organizar o código em componentes independentes, testáveis e reutilizáveis.*
- *Este padrão de arquitetura é uma metodologia de design de software que enfatiza a separação de interesses e separa a aplicação em módulos distintos.*

Programação Web – 2024/2025 – LEI ISEC/DEC - JB

.NET CORE 8



O principal objetivo da Arquitetura Limpa é criar uma estrutura que facilite a gestão e a manutenção de uma aplicação à medida que esta cresce e muda com o tempo.

- ***Também torna mais fácil adicionar novas funcionalidades, corrigir bugs e fazer alterações nas funcionalidades existentes sem afetar o resto da aplicação.***

ProgramaçãoWeb – 2024/2025 – LEI DEISI/ISEC/IPC – JB

.NET CORE 8

O principal objetivo da Arquitetura Limpa é criar uma estrutura que facilite a gestão e a manutenção de uma aplicação à medida que esta cresce e muda com o tempo.

- ***Também torna mais fácil adicionar novas funcionalidades, corrigir bugs e fazer alterações nas funcionalidades existentes sem afetar o resto da aplicação.***

ProgramaçãoWeb – 2024/2025 – LEI DEI/ISEC/IPC – JB

.NET CORE 8

2.7. Micro-Frontends e Micro Serviços

À medida que as aplicações web crescem em complexidade, tem havido um impulso significativo para a adoção de arquiteturas escaláveis e de fácil manutenção.

Os Micro-Frontends surgiram como uma solução poderosa para este desafio.

Basicamente, uma arquitetura de micro-frontend divide o front-end de uma aplicação web em partes mais pequenas e mais geríveis, muito semelhante à forma como os microsserviços dividem as aplicações de back-end.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEC – JB

.NET CORE 8

Um Micro Frontend é a contraparte frontend dos micro-serviços. Tal como os microsserviços dividem os monólitos de back-end em serviços fracamente acoplados, os micro-frontends dividem uma grande aplicação web em partes mais pequenas e independentes que funcionam em conjunto sem problemas para formar uma aplicação completa.

- *Com os micro frontends, cada parte da interface do utilizador (IU) pode ser desenvolvida e mantida separadamente. As equipas podem trabalhar de forma autónoma, garantindo lançamentos mais rápidos, melhor escalabilidade e desempenho melhorado.*

Programação Web – 2024/2025 – LEI+CE+... – ISEC/IPC - ...

.NET CORE 8

Micro Frontend vs. Microserviços

Enquanto os microsserviços se concentram em dividir a funcionalidade de back-end em serviços independentes, os micro-frontends aplicam este conceito à interface do utilizador.

- *Os microsserviços lidam com funcionalidades de backend (operação de base de dados, APIs).*
- *Os Micro Frontends gerem a UI, tratando da interação do utilizador e da camada de apresentação.*

Programação Web 2024/2025 – LEI+CE+PL – ISEC/IPC – JB

.NET CORE 8

Objectivos comuns:

- *Acoplamento fraco: cada micro-serviço/microfrontend funciona de forma independente.*

- *Autonomia: As equipas podem desenvolver e implementar sem terem de coordenar com outras equipas.*

- *Escalabilidade: cada serviço ou front-end pode ser dimensionado de forma independente para satisfazer a procura do utilizador.*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/nFC - JB

.NET CORE 8

Quando utilizar a arquitetura Micro Frontend

Os micro front-ends são mais adequados para aplicações grandes e complexas que exigem modularidade e desenvolvimento independente. Eis os principais cenários onde a sua utilização é recomendada:

1. Aplicações grandes e complexas: Os micro front-ends ajudam a dividir as aplicações grandes em partes geríveis, melhorando a organização e reduzindo o tempo de construção.

Programação Web 2024/2025 – LEI+CE+PL – ISEC/DEC - JB

.NET CORE 8

2. Várias equipas: as equipas podem trabalhar de forma independente em diferentes partes da aplicação, permitindo o desenvolvimento paralelo e implementações independentes.

3.º Implementações independentes: Os micro front-ends permitem que recursos específicos sejam atualizados sem voltar a implementar a aplicação inteira, reduzindo os riscos de implementação.

Programação Web - 2024/2025 – LEI DI-CE+PL – ISEC/IPC – JB

.NET CORE 8

4. Diferentes pilhas de tecnologia: Diferentes partes da aplicação podem utilizar diferentes estruturas, permitindo flexibilidade nas escolhas tecnológicas.

5. Escalabilidade: Os micro frontends são mais fáceis de escalar e manter, uma vez que cada parte da aplicação é isolada e pode crescer de forma independente.

Programação Web – 2024/2025 – LEI D+CC+PL – ISEC/IPC

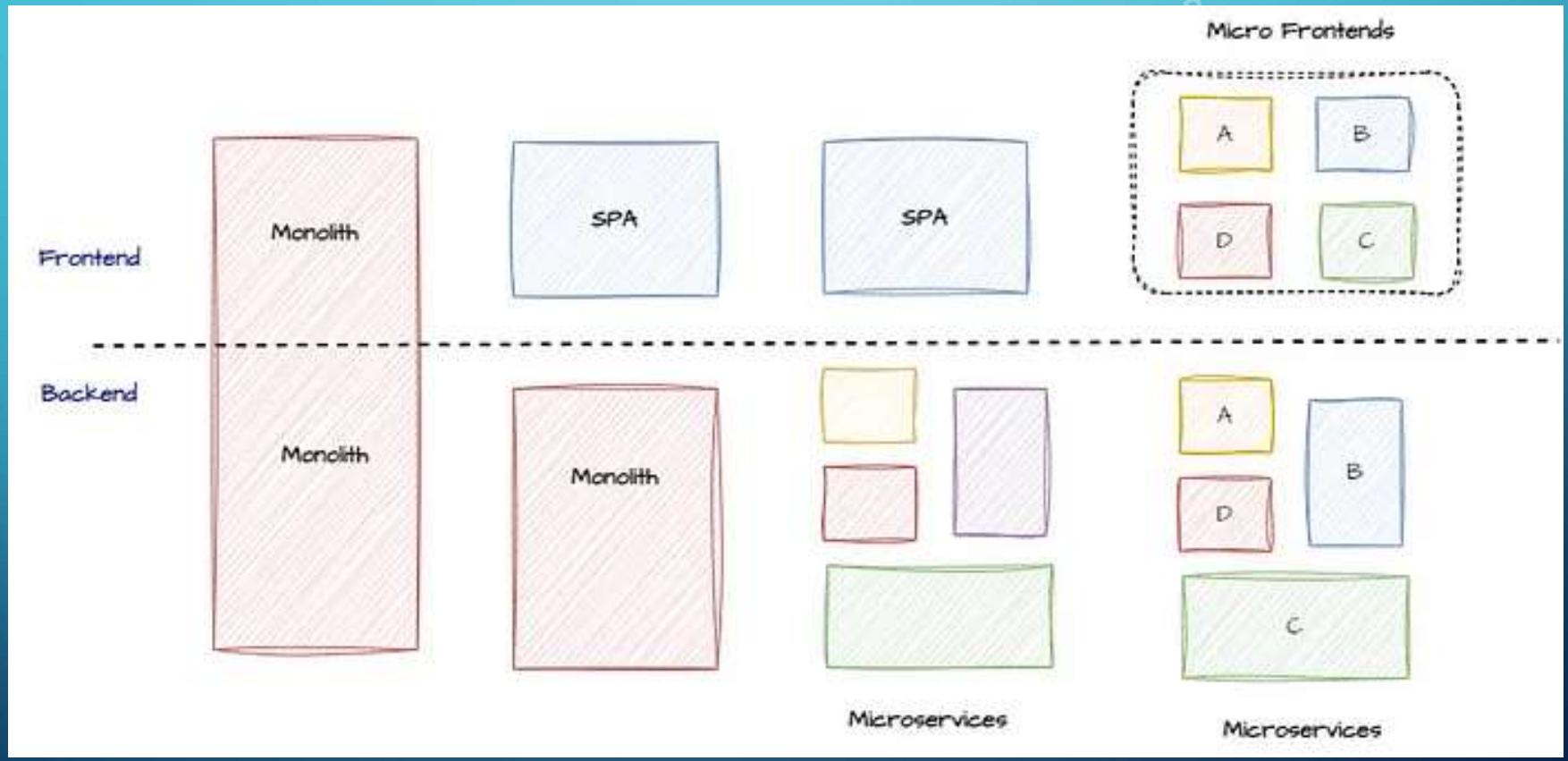
.NET CORE 8

6.° Modernização de legados: Substituir os sistemas legados de forma incremental sem uma reescrita completa, permitindo uma transição gradual para as tecnologias modernas.

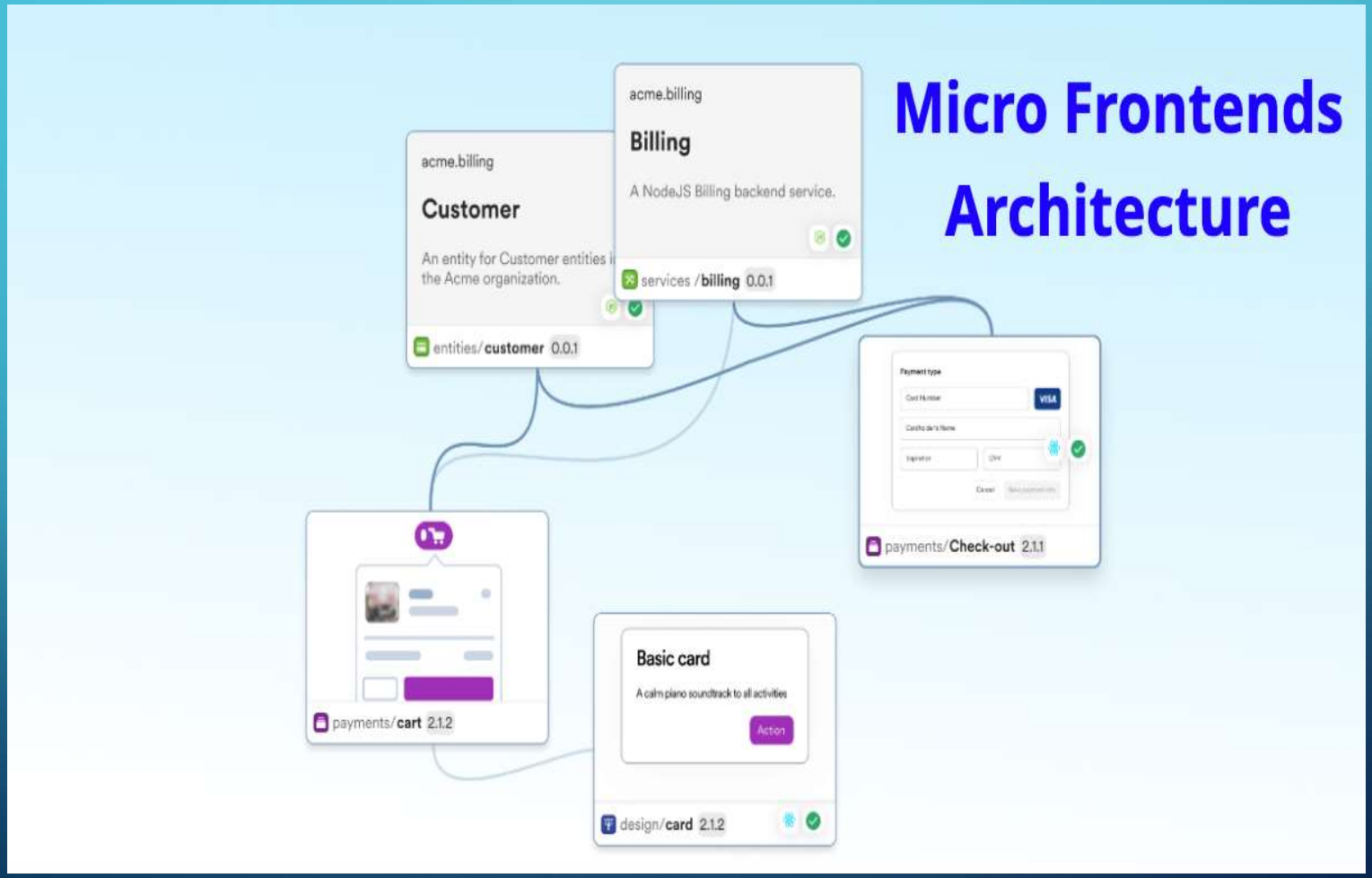
7. Melhorias de desempenho: otimize os tempos de carregamento armazenando em cache ou carregando lentamente diferentes partes da aplicação.jectivos comuns:

Programação Web – 2024/2025 – LEI D+GE+PL – ISEC/IPC – JB

.NET CORE 8


Micro Frontends
A
B
D
C
Frontend
Monolith
SPA
SPA
Backend
Monolith
Monolith
Microservices
A
B
D
C
Microservices

.NET CORE 8


Micro Frontends Architecture
acme.billing
Billing
A NodeJS Billing backend service.
services /billing 0.0.1
acme.billing
Customer
An entity for Customer entities
the Acme organization.
entities/customer 0.0.1
payments/Check-out 2.1.1
payments/cart 2.1.2
Basic card
A calm piano soundtrack to all activities
Action
design/card 2.1.2

.NET CORE 8

Requisitos principais para a arquitetura micro frontend

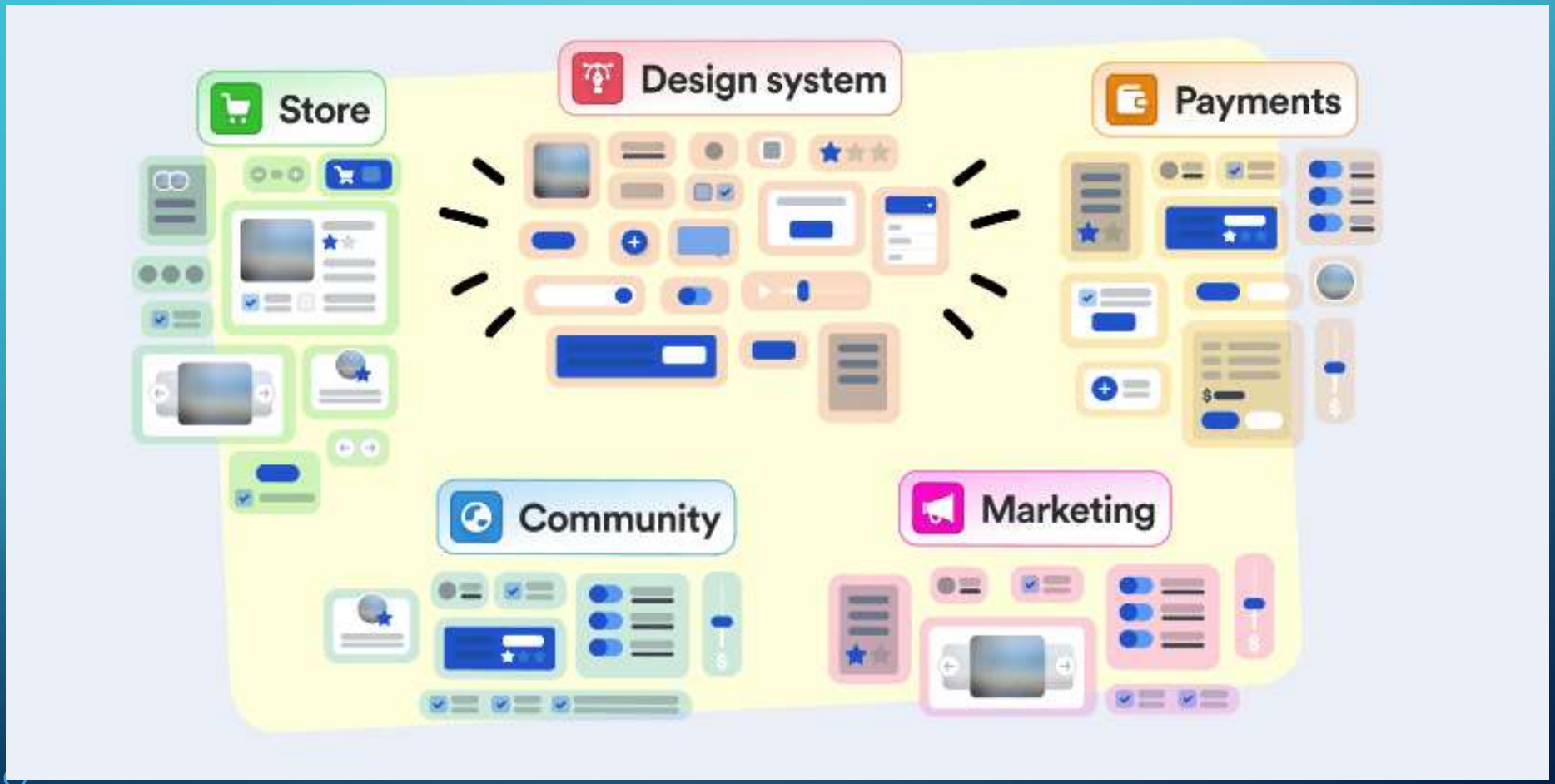
Para implementar microfrontends com sucesso num ambiente empresarial, devem ser cumpridos vários requisitos críticos:

1. Desenvolvimento desacoplado

O desenvolvimento desacoplado na arquitetura de micro frontend permite que cada micro frontend seja desenvolvido, implementado e mantido de forma independente. Cada um, representando uma característica distinta, pode utilizar a sua própria pilha de tecnologia, cronograma de lançamento e processos.

Programação Web – 2024/2025 – TP, D+CE+PL – ISEC/IPC - JB

.NET CORE 8



.NET CORE 8

*Esta **autonomia** reduz as dependências da equipa, permitindo uma inovação e iteração mais rápidas.*

Nas grandes empresas, os front-ends tornam-se muitas vezes monolíticos e difíceis de gerir, onde as alterações podem ter impacto em todo o sistema, atrasando o desenvolvimento e aumentando os riscos.

O desenvolvimento desacoplado divide o frontend em partes geríveis, permitindo que as equipas:

Programação Web – 2024/2025 – TPSI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

- *Inovar mais rapidamente: escolha ferramentas e frameworks ideais, acelerando o desenvolvimento.*
- *Reduza os riscos: isole as alterações para minimizar o impacto na aplicação mais ampla.*
- *Escale eficientemente: escale de forma independente cada microfrontend, apoiando o crescimento orgânico.*

Programação Web – 2024/2025 – LEI D+CE+... – ISEC/IPC - JB

.NET CORE 8

- 2. Controlo de versão e gestão de dependências
  - O controlo de versões e a gestão de dependências são aspetos cruciais da arquitetura micro frontend. Numa configuração de micro frontend, cada módulo ou componente de frontend é desenvolvido, versionado e mantido de forma independente.
  - Esta independência significa que cada módulo pode evoluir ao seu próprio ritmo, com as suas dependências e sistema de versionamento específicos.

Programação Web 2024/2025 – LEI D+CE PL – ISEC/IPC - JB

.NET CORE 8

- *O controlo de versão por componente garante que as atualizações num micro frontend não interrompem inadvertidamente outras partes da aplicação, enquanto a gestão eficaz de dependências evita conflitos e garante que cada módulo pode funcionar como pretendido no ecossistema mais amplo.*
- *Em aplicações de grande escala, a gestão de dependências e versões torna-se cada vez mais complexa à medida que são introduzidos mais microfrontends.*

Programação Web - 2024/2025 – LEI+CE+I – ISEC/DEC - JB

.NET CORE 8

- *Sem um sistema robusto, é fácil surgirem conflitos de dependências, levando a problemas como bibliotecas duplicadas, incompatibilidades de versões ou até mesmo falhas de aplicações.*
- *O controlo de versão adequado e a gestão de dependências garantem que:*
  - *A estabilidade é mantida: as atualizações num micro frontend não prejudicam os outros, mantendo a estabilidade geral da aplicação.*

Programação Web – 2024/2025 – LEI D+CE+P – ISEC/DEC - JB

.NET CORE 8

- *As dependências são consistentes: cada microfrontend utiliza as versões corretas das bibliotecas partilhadas, evitando conflitos e redundância.*

- *Flexibilidade de implementação: as equipas podem atualizar ou reverter microfront-ends específicos sem afetar toda a aplicação, permitindo estratégias de implementação mais flexíveis e responsivas.*

Programação Web – 2024/2025 – LEI DEE+PL – ISEC/IPC – JB

.NET CORE 8

Porque utilizar micro-front-ends?

O front-end monolítico tradicional pode tornar-se um estrangulamento à medida que a aplicação cresce. Lidar com grandes bases de código, gerir dependências e coordenar equipas tornam-se tarefas árduas. Os micro-Frontends resolvem estes problemas:

- *Escalabilidade: Diferentes equipas podem trabalhar em diversas partes da aplicação em simultâneo, sem interferir umas com as outras.*

Programação Web 2024/2025 – LEI DI-CE-ISEC/PC - JB

.NET CORE 8

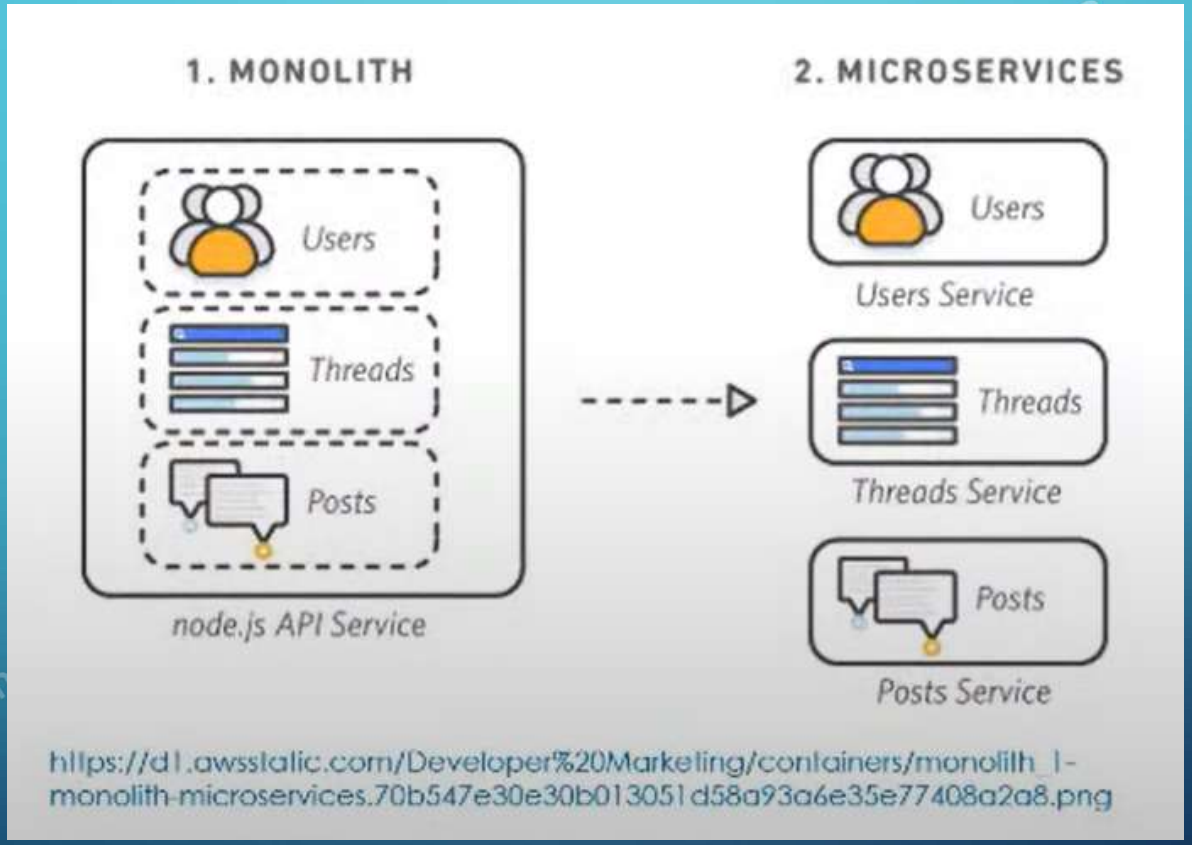
- *Capacidade de manutenção: as bases de código mais pequenas são inerentemente mais fáceis de gerir e atualizar.*

- *Agnóstico em tecnologia: as equipas podem escolher a pilha tecnológica mais adequada para o seu módulo específico.*

- *Independência da implementação: cada micro-frontend pode ser implementado de forma independente, reduzindo os riscos de implementação e melhorando a velocidade de implementação.*

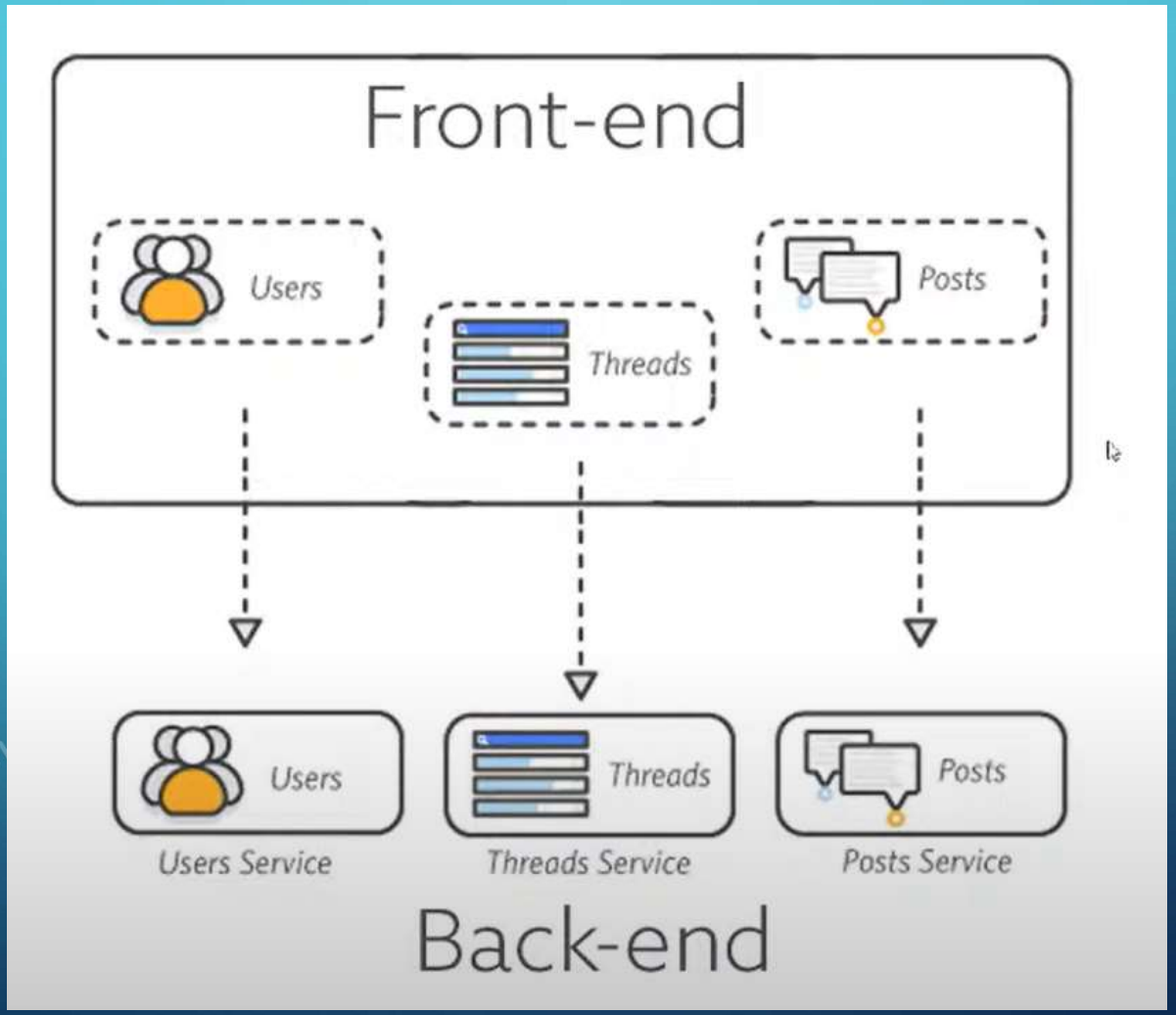
Programação Web – 2024/2025 – LEI LA-CE+PL – ISEC/ISC – JB

.NET CORE 8



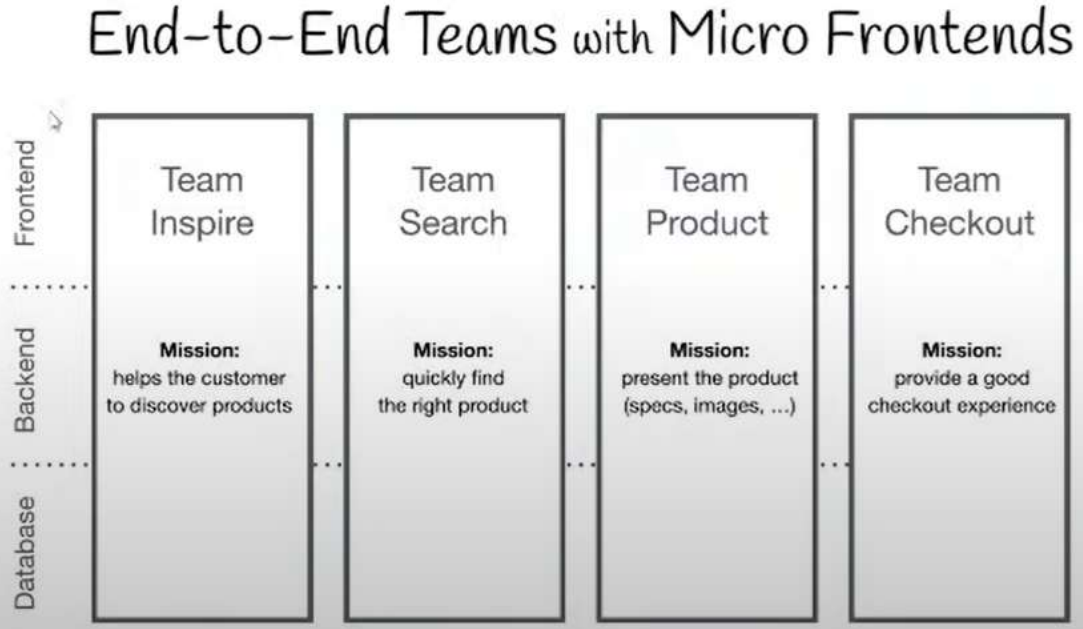
https://d1.awsstatic.com/Developer%20Marketing/containers/monolith_1-monolith-microservices.70b547e30e30b013051d58a93a6e35e77408a2a8.png

.NET CORE 8


Front-end
Users
Threads
Posts
Users
Threads
Posts
Users Service
Threads Service
Posts Service
Back-end

.NET CORE 8

"A web application is broken up by its pages and features, with each feature being owned end-to-end by a single team" – ThoughtWorks



.NET CORE 8


The Model Store
basket: 0 item(s)
Related Products
Tractor Porsche-Diesel Master 419
buy for 66,00 €
Team Product
Team Checkout
Team Inspire

.NET CORE 8

Conceitos Básicos e Arquitetura

- Modularidade e Composição
  - Uma arquitetura Micro-Frontend baseia-se nos princípios de modularidade e composição. Cada módulo deve encapsular a sua funcionalidade e ser implantável de forma independente. Além disso, estes módulos podem ser compostos em conjunto para formar a aplicação completa.

Programação Web – 2024/2025 – LEI D+CL+RL – ISEC/IPC - JB

.NET CORE 8

Padrões e Técnicas para implementar Micro-Frontends:

- Composição do lado do servidor: O servidor compõe os fragmentos de frontend e apresenta-os como uma resposta única ao cliente.

- Composição do lado do cliente: O cliente procura vários fragmentos e compõe-nos dinamicamente.

- Edge-Side Included (ESI): Combina fragmentos na camada de cache, levando a um melhor desempenho.

Programação Web – 2024/2025 – LEI D+CSI+PL – ISEC/IPC - JB

.NET CORE 8

Desafios e Considerações

A adoção de Micro-Frontends tem os seus desafios:•

Desempenho: a composição de vários fragmentos de frontend pode introduzir latência.

- **Comunicação**: É crucial garantir uma comunicação eficaz e um fluxo de dados entre fragmentos.
- **Consistência**: Manter uma aparência consistente em todos os fragmentos pode ser um desafio.
- **Gestão de versões**: Fragmentos diferentes podem ter dependências de versões diferentes de bibliotecas, levando a possíveis conflitos.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – JB

.NET CORE 8

- *Em Resumo:*

Os micro-Frontends representam uma mudança significativa na forma como arquitetamos as aplicações web modernas.

Ao dividir o front-end em partes mais pequenas e geríveis, as equipas podem alcançar maior escalabilidade, capacidade de manutenção e flexibilidade.

É muito importante ponderar os benefícios em relação aos motenciais desafios e considerar factores como o desempenho, a comunicação e a consistência.

Programação Web - 2024/2025 – LEI-D+CE+TP – ISEC/PC - JB

.NET CORE 8

2.8. API

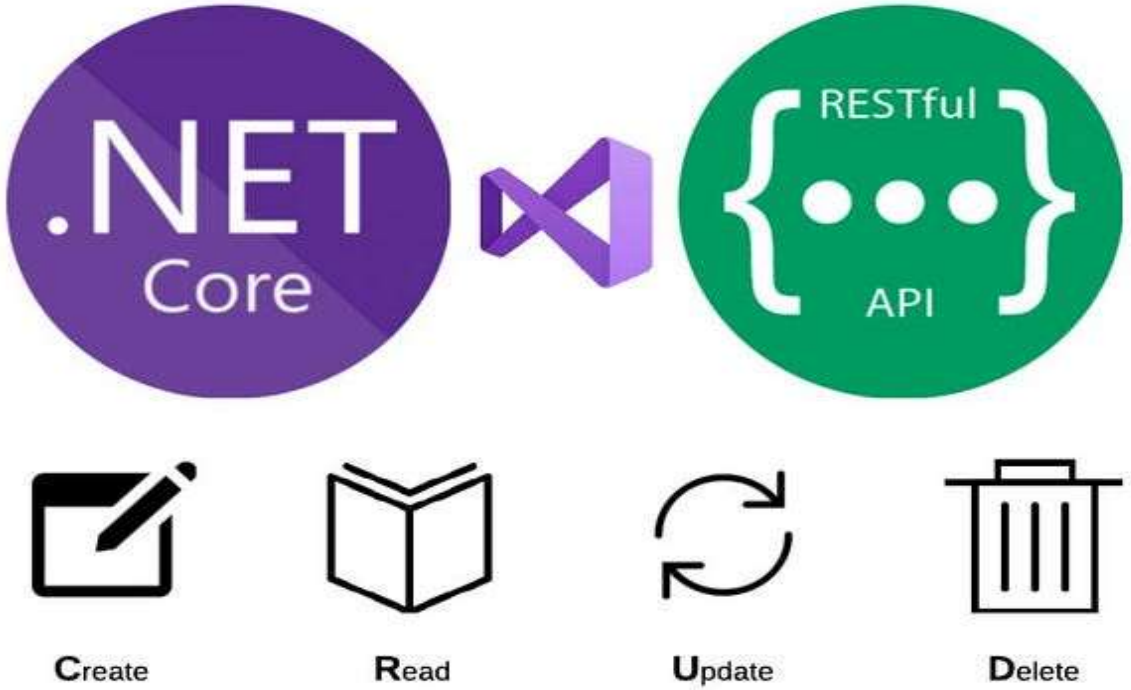
O que é uma Web API ?

Uma Web API (*Application Programming Interface*) é como um mensageiro que permite que diferentes aplicações de software comuniquem e partilhem informação através da Internet.

- Consiste num conjunto de regras e protocolos que possibilitam a comunicação entre aplicações.
- As APIs Web são normalmente utilizadas para criar aplicações Web e móveis, facilitando a ligação a serviços, a recuperação de dados e a executar de ações.

Programação Web – 2024/2025 – LEI+CET+PL – ISEC/IPC - JB

.NET CORE 8



.NET CORE 8

As API, tornam mais fácil a criação de aplicações que possam comunicar entre si, partilharem informações e trabalhar juntos de uma forma integrada.

- As APIs Web são normalmente criadas utilizando tecnologias Web, como HTTP (Hypertext Transfer Protocol) e JSON (JavaScript Object Notation) para permitir a troca de dados entre aplicações clientes (como browsers Web ou aplicações móveis) e aplicações ou serviços do lado do servidor.

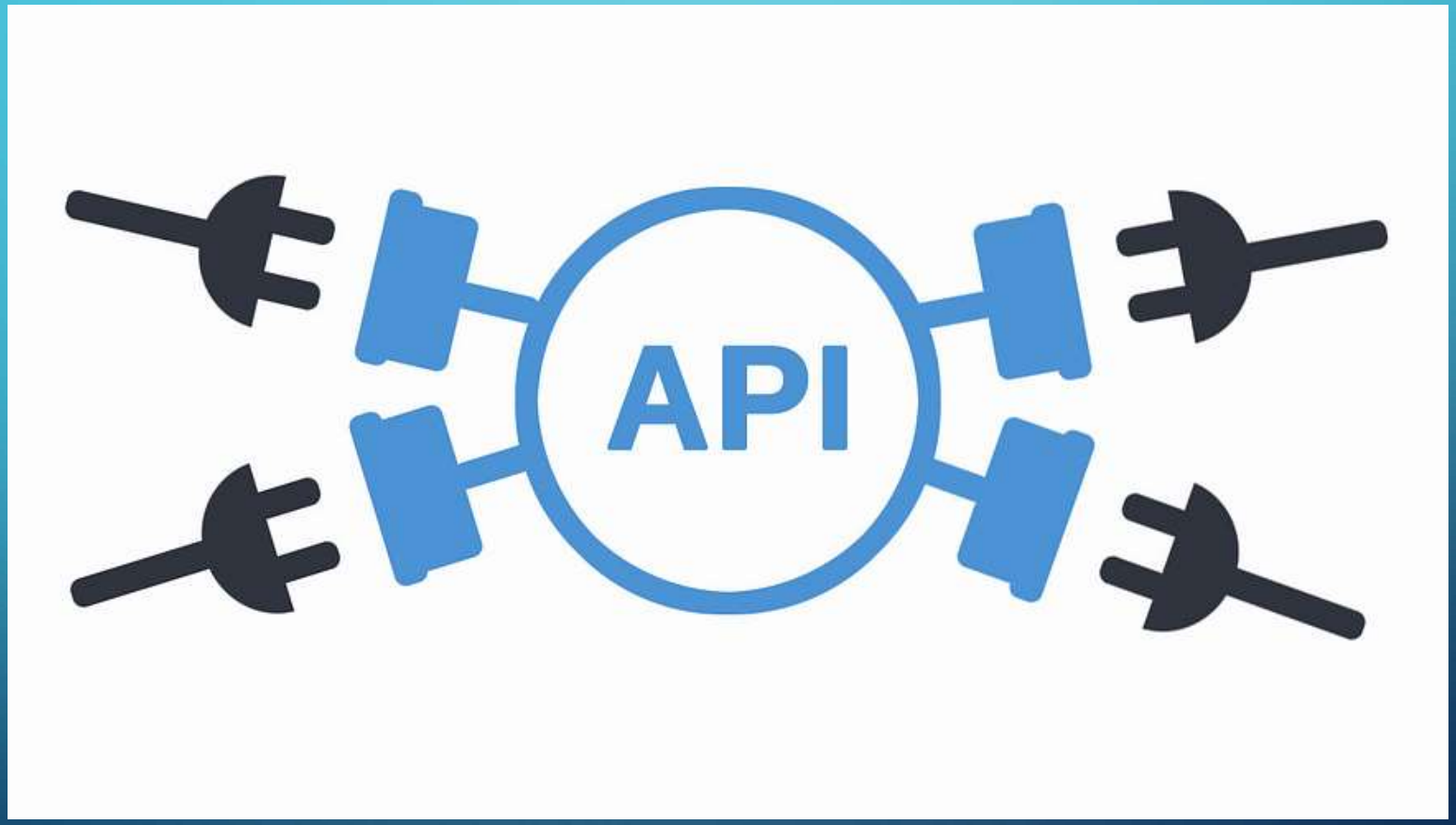
Programação Web – 2024/2025 – LEI D+CE+RL – ISEC/IPC - JB

.NET CORE 8

- As interfaces de programação de aplicações (APIs) são fundamentais no desenvolvimento de software moderno.
- Permitem que diversas aplicações comuniquem e partilhem dados de forma integrada, possibilitando a integração eficaz de diferentes sistemas e serviços.
- No desenho e implementação de API é essencial seguirem-se bons princípios de design de API pois é crucial criarem-se interfaces robustas, escaláveis e fáceis de utilizar.

Programação Web 2024/2025 – LEI+CE+TW – ISEC/AnC - JB

.NET CORE 8



.NET CORE 8

- É habitual construírem-se as chamadas API Web RESTful com ASP.NET Core em .NET 8.0, integranda o Entity Framework Core.
- Para testar a API, é também habitual utilizar-se o Swagger.
- Para a persistência dos dados, utiliza-se qualquer SGBD
  - Nas aulas utilizaremos SGBD Microsoft SQL Server, para a gestão e armazenamento de dados.

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/DEC - JB

.NET CORE 8

O que é o REST

- REST (Representational State Transfer) é um estilo arquitetural para o desenvolvimento de APIs RESTful, facilitando a comunicação padronizada entre sistemas informáticos na web.
  - Opera num modelo cliente-servidor, permitindo a evolução independente das aplicações sem interdependências.

Programação Web – 2024/2025 – LEI D+GJ+PL – ISEC – JPC - JB

.NET CORE 8

- O REST é comummente associado ao HTTP, onde quatro verbos principais governam as interações com os recursos:
  1. **GET**: Permite obter um recurso específico (por id) ou uma coleção de recursos.
  2. ° **POST**: Cria um novo recurso.
  3. ° **PUT**: Atualiza um recurso específico (por id).
  4. ° **DELETE**: Remove um recurso específico por id.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

Tipos de APIs

1. REST (Transferência de Estado Representacional):

- Utiliza métodos HTTP padrão.
- Arquitetura sem estado.
- Recursos identificados por URLs.
- Amplamente utilizado devido à simplicidade e escalabilidade.

Programação Web 2024/2025 – LEI DI-CE+PL – ISEC/IPC - JB

.NET CORE 8

2.º SOAP, Simple Object Access Protocol, (Protocolo Simples de Acesso a Objectos)

- Protocolo para troca de informação estruturada.
- Depende de XML.
- Suporta operações complexas e maior segurança.
- Utilizado em aplicações de nível empresarial.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

3. GraphQL

- Permite aos clientes solicitar exatamente os dados de que necessitam.
- Reduz a pesquisa excessiva e insuficiente de dados.
- Suporta consultas mais flexíveis em comparação com o REST

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

4. gRPC

- Utiliza HTTP/2 para transporte e buffers de protocolo para serialização de dados.
- Suporta streaming bidirecional.
- Elevado desempenho e adequado para microsserviços.

Programação Web – 2024/2025 – LEI DTSE+PL – ISEC/IPC - JB

Princípios Básicos de Design de API

1. Consistência

- A consistência é a chave para uma API bem concebida. A API deve ser consistente na sua estrutura, convenções de nomenclatura e tratamento de erros.
  - Por exemplo, deve-se utilizar convenções de nomenclatura semelhantes para os end points.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

(Consistência)

- Aplicar formatos uniformes para respostas e erros
- Uniformize os nomes dos parâmetros e os tipos de dados

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



.NET CORE 8

2. Statelessness

- A API deve ser concebida para não ter estado.
- Cada pedido de um cliente deve conter todas as informações necessárias para processar o pedido.
  - Isto simplifica o design do servidor e melhora a escalabilidade.
  - O Statelessness significa que o servidor não armazena qualquer contexto de cliente entre pedidos, o que ajuda na distribuição da carga por vários servidores.

Programação Web – 2024/2025 – LEI D+G+PL – ISEC /IPC - JB

.NET CORE 8

3. Design Orientado a Recursos

- Tudo na API deve ser considerado como um recurso.
  - Os recursos podem ser objetos, dados ou serviços, e cada um deve ter um identificador único (normalmente um URL em APIs RESTful).
  - Os endpoints devem ser concebidos para representar recursos e utilizarem métodos HTTP para executar as ações

Programação Web – 2024/2025 – LEI DEIS+PL – ISEC/IPC - JB

4.º Utilizar métodos HTTP padrão

A convenção de métodos HTTP para realizar operações sobre recursos é:

- **GET** para recuperação de recursos.
- **POST** para criação de recursos.
- **PUT** para atualização de funcionalidades.
- **DELETE** para eliminar recursos.

A utilização destes métodos padrão torna a API intuitiva e fácil de usar.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

5. Versionamento

Deve-se incluir o controlo de versão no design da API para lidar com atualizações sem perturbar os clientes existentes.

As estratégias comuns de versionamento incluem:

- Versionamento de URL (/v1/resource).
- Versionamento de cabeçalho (Aceitar: application/vnd.yourapi.v1+json).
- Versionamento de parâmetros (/resource?version=1).

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

Autenticação e Autorização

A API pode ser protegida usando autenticação e autorização

Os métodos comuns incluem:

- **OAuth:** Um padrão aberto amplamente utilizado para delegação de acesso, normalmente utilizado para autenticação baseada em tokens.
- **JWT (JSON Web Tokens):** Tokens que codificam uma carga útil com uma assinatura para garantir a integridade dos dados.
- **API Keys:** Tokens simples passados através de cabeçalhos HTTP ou parâmetros de consulta para autenticar pedidos.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

2.9. Minimal API

O que é uma Minimal API ?

No domínio do desenvolvimento web, a simplicidade resulta frequentemente em eficiência e velocidade.

A introdução das Minimal API no ASP.NET Core, representam uma metodologia inovadora que altera substancialmente o modo de criação de API com a esta filosofia minimalista.

- Com Minimal API, não é necessário recorrer-se à abordagem tradicional de scaffolding, adotando-se um método sucinto de definição de rotas e ações da API.

Programação Web - 2024/2025 - LEI D+CTeSP - ISEC/IPC - JB

.NET CORE 8

As APIs mínimas representam uma mudança fundamental no desenvolvimento do ASP.NET Core, defendendo uma abordagem de regresso ao básico que enfatiza a clareza e a brevidade.

- As APIs mínimas capacitam os programadores para articular rotas e ações de API com uma simplicidade incomparável.

Programação Web – 2024/2025 – LEI-DI-CE+01 – ISEC/IPC

.NET CORE 8

Para se perceber as diferenças resultantes da utilização de Minimal API, vejamos um exemplo prático

```
var aplicacao = WebApplication.Create(args);
app.MapGet("/", () => "Olá mundo!");
app.Run();
```

- Neste excerto, criou-se Minimal API que mostra aos utilizadores a mensagem “Hello World!” mensagem quando acedida no endpoint raiz (“/”).
- Com apenas algumas linhas, este exemplo resume a essência das Minimal API.

.NET CORE 8

As Minimal API destacam-se não só pela sua simplicidade, mas também pelo tratamento da complexidade, principalmente quando se trata de gerir parâmetros e configurações de rotas complexas.

```
var construtor =
WebApplication.CreateBuilder(args);
var app=construtor.Build();
app.MapGet("/alunos/{alunoId}/cursos/{cursoId}",
 (int alunoId, int cursoId) => $"O ID do aluno é
{alunoId} e o ID do curso é {cursoId}");
app.Run();
```

Mapeamento de rotas nas Minimal API

- **MapGet**
  - Mapeia uma rota para um pedido GET para obter um registo.
- **MapPost**
  - Mapeia uma rota para um pedido POST para inserir um registo.
- **MapPut**
  - Mapeia uma rota para um pedido PUT para atualizar um registo.

Programação Web – 2024/2025 – LEI D+CB+RL – ISEC/IPC - JB

.NET CORE 8

- **MapPatch**
  - Mapeia uma rota para um pedido PATCH para atualizar um registo.
- **MapDelete**
  - Mapeia uma rota para um pedido DELETE para eliminar um registo.
- **MapMethods**
  - Mapeia uma rota para qualquer outro método ou métodos HTTP, por exemplo, CONNECT ou HEAD.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

Mapeamento de parâmetros

Opcionalmente, podem-se utilizar atributos para definir a partir dos quais as Minimal API devem definir os valores dos parâmetros:

[FromServices]

- O parâmetro será definido a partir dos serviços de dependência registados.

[FromRoute]

- O parâmetro será definido a partir de um segmento de rota

Programação Web - 2024/2025 – LEI D+CE+RAS – ISEC/IPC - JB

.NET CORE 8

[FromQuery]

- O parâmetro será definido a partir de um parâmetro da string de consulta

[FromBody]

- O parâmetro será definido a partir do corpo do pedido HTTP.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

2.10. Injecção de Dependência

O que é a injeção de dependências?

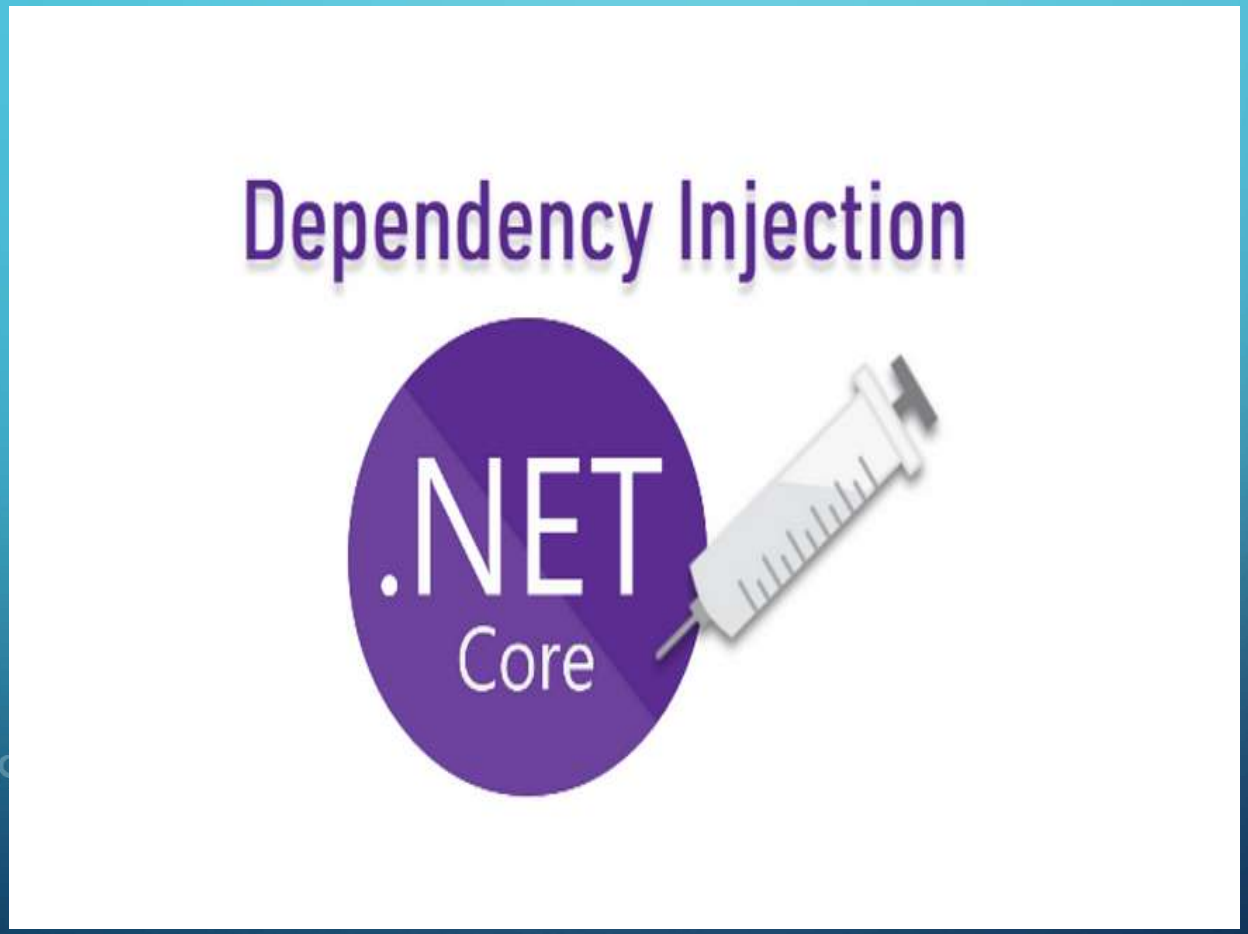
A injeção de dependências é uma técnica que nos permite dissociar a implementação de um objeto das suas dependências.

Podemos separar a implementação de um objeto das suas dependências com a injeção de dependências. Em vez de um objeto criar as suas dependências, estas são fornecidas ao objeto a partir do exterior.

- Desta forma, podemos alterar ou trocar dependências sem mexer no código do objeto.

Programação Web – 2024/2025 – LEI DI CE+PL – ISEC/IPC - JB

.NET CORE 8



.NET CORE 8

- Testar objetos de forma independente é muito mais simples porque podemos utilizar objetos simulados em vez de objetos reais.

O ASP.NET Core possui uma estrutura de injeção de dependências integrada para fornecer serviços a diferentes componentes da aplicação.

- É utilizado em todos os lugares para ligar outras partes da aplicação aos serviços necessários.
- A estrutura utiliza um contentor de Inversão de Controlo, Inversion of Control. (IoC) para lidar com todas as dependências entre os componentes.

Programação Web – 2024/2025 – LEI D-TSE+PL – ISEC/IPC - JB

.NET CORE 8

- Inversion of Control Container, IoC
- Um contentor IoC é uma estrutura que gere a criação e a vida útil dos objetos numa aplicação. Encarrega-se de criar e monitorizar objetos numa aplicação.
  - Além disso, fornece uma forma de registar e resolver dependências quando necessário.
  - No ASP.NET Core, o contentor IoC está integrado na estrutura e pode ser configurado no ficheiro Program.cs.

Programação Web – 2024/2025 – LEI DI-CE+PL ISEC/IPC - JB

.NET CORE 8

- Inversion of Control Container, IoC

No exemplo abaixo, está-se a registar um serviço chamado *UserService* que implementa a interface *IUserService*.

```
// Program.cs
builder.Services.AddScoped<IUserService, UserService>();
```

- O método **AddScoped** indica ao contentor para criar uma nova instância da classe **UserService** para cada pedido do cliente.
- Estão disponíveis outros métodos para registar serviços com tempos de vida diferentes, como o **AddSingleton** e o **AddTransient.**
- Depois de um serviço ser registado num IoC, pode ser injetado noutros componentes da aplicação usando injeção de construtor

.NET CORE 8

- Injeção de Construtor
- A injeção de construtores é uma técnica em que são fornecidas dependências a uma classe através do seu construtor. Isto facilita a visualização das dependências de uma classe e permite que o contentor IoC crie e injete as dependências necessárias automaticamente.
- public class UserController: ControllerBase
- {
- private read-only IUserService _userService;

Programação Web – 2024/2025 LEI D+PL+ISEC/AC - JB

.NET CORE 8

- Injeção de Construtor - Exemplo

```
public class UserController: ControllerBase
{
   private read-only IUserService _userService;
   Public UserController (IUserService userService)
   {
     _userService = userService;
   }
}
```

.NET CORE 8

- Injeção de Construtor

- **No exemplo mostrado, a classe *UserController* depende da interface *IUserService*.**
  - **A dependência é injetada através do construtor e o contentor IoC cria e injeta automaticamente a dependência necessária.**

Programação Web – 2024/2025 – EESD+CET+PL – ISEC/IPC - JB

.NET CORE 8

- **Service Life Time:**
  - **Singleton,**
  - **Scoped,**
  - **Transient**

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

- **Singleton**
- **O método *AddSingleton* regista um serviço com o contentor de injeção de dependências como um *singleton*.**
  - **Um *singleton* é um objeto criado apenas uma vez e a mesma instância é partilhada por toda a aplicação.**
    - **Isto significa que qualquer componente que dependa do serviço singleton registado receberá a mesma instância do serviço.**

Programação Web – 2024/2025 – LEI D+CE+... – ISEC/JBC - JB

.NET CORE 8

- Injeção de Construtor

```
construtor.Services.AddSingleton<IEmailService, EmailService>();
```

- **No código acima, estamos a registar um serviço chamado EmailService que implementa a interface IEmailService. O método AddSingleton indica ao contentor para criar apenas uma instância da classe EmailService e partilhá-la em toda a aplicação.**
- **O Singleton é útil quando queremos partilhar dados ou recursos entre diferentes componentes de**

Programação Web – 2024/2025 – LEI+CE-PL – ISEC

.NET CORE 8

- **O Singleton é útil quando queremos partilhar dados ou recursos entre diferentes componentes de uma aplicação.**
  - **No exemplo acima, temos um EmailService que queremos partilhar em diferentes partes da nossa aplicação.**
  - **Ao registar o EmailService como singleton, podemos garantir que todos os componentes que necessitam de enviar um email utilizarão a mesma instância do serviço.**

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC – JB

.NET CORE 8



- Um serviço ***singleton*** é criado quando a aplicação é iniciada e não é descartada até ser encerrada.
  - Isto significa que quaisquer recursos utilizados pelo serviço singleton são mantidos na memória durante a vida útil da aplicação.
- Portanto, devemos ter cuidado ao utilizar singletons para evitar desperdícios de memória ou problemas de desempenho.

Programação Web – 2024/2025 – LEI+LCE+PL – ISEC/IPC

.NET CORE 8

- **Devemos utilizar o método *AddSingleton* com moderação e apenas para serviços que necessitam de ser partilhados por toda a aplicação.**

  - **Para outros serviços, como os específicos para um pedido específico, devemos utilizar os métodos *AddScoped* ou *AddTransient*.**

Programação Web – 2024/2025 – LEI DEI-ISEC/ISEC – JB

.NET CORE 8

- **Scoped**
- **O método *AddScoped* é utilizado para registar um serviço com o contentor de injeção de dependências como um scoped service.**
  - **Um scoped serviço é criado uma vez por pedido do cliente (tempo de vida com âmbito definido) e partilhado dentro desse pedido.**
  - **Desse modo, qualquer componente que dependa do scoped service registado receberá a mesma instância do serviço num único pedido do cliente.**

Programação Web - 2024/2025 - LEI D+CE+PL - ISEC/IPC - JB

.NET CORE 8

- Os *scoped services* são úteis ao partilhar dados ou recursos numa única solicitação do cliente.
  - Por exemplo, temos um contexto de base de dados que queremos utilizar para um único pedido de cliente.
  - Ao registar o contexto da base de dados como um *scoped service*, podemos garantir que todos os componentes que utilizam o contexto num único pedido do cliente utilizam a mesma instância do contexto.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – JB

.NET CORE 8

```
construtor.Services.AddDbContext<DataContext>(opt =>
opt.UseSqlServer(builder.Configuration.GetConnectionString("DataContext")));
```

- No exemplo acima, o Entity Framework DbContext está registado por defeito como um *scoped service*
- É importante referir que os *scoped services* são descartados no final de cada pedido do cliente.
  - Isto significa que quaisquer recursos utilizados pelo scoped service são libertados no final de cada pedido, o que ajuda a evitar desperdícios de memória e melhora o desempenho.
  - Devemos utilizar o método AddScoped para serviços que precisam de ser partilhados numa única solicitação do cliente.

Programação Web 2024/2025 – LEI D+CE-PL – ISEC/II

Transient

- O método *AddTransient* regista um serviço com o contentor de injeção de dependências como um serviço transitório.
  - Um *Transient Service* é criado sempre que solicitado e não é partilhado entre diferentes componentes ou pedidos.
  - Isto significa que qualquer componente que dependa do *transient service* registado receberá uma nova instância do serviço sempre que for solicitado.

Programação Web – 2024/2025 – LEI D+G+PL – ISEC/IPC - JB

.NET CORE 8

```
construtor.Services.AddTransient<IPasswordGeneratorService,
PasswordGeneratorService>();
```

- Neste exemplo, registou-se um serviço designado por *PasswordGeneratorService* que implementa a interface *IPasswordGeneratorService*. O método AddTransient indica ao contentor para criar uma nova instância da classe PasswordGeneratorService sempre que solicitado.
- Os *Transient Services* são úteis quando queremos criar uma nova instância de um serviço sempre que necessário.
- Ao registar o PasswordGeneratorService como um Transient Service, podemos garantir que cada vez que o serviço é solicitado, será criada uma nova instância do gerador de passwords.

Programação Web - 2024/2025 - LEI - LCF+I+L - ISEC/IPC

.NET CORE 8

- Devemos utilizar o método ***AddTransient*** para os serviços que necessitam de ser criados sempre que solicitados.

- Para serviços que necessitam de ser partilhados num único pedido do cliente, devemos utilizar o método ***AddScoped***

- Para serviços que necessitam de ser partilhados por toda a aplicação, devemos utilizar o método ***AddSingleton***.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – JB

.NET CORE 8

- **Como Decidir qual o Service Lifetime a utilizar ?**
  - **A decisão de utilizar o *AddSingleton*, *AddTransient* ou *AddScoped* para registar um serviço com o contentor de injeção de dependências depende do tempo de vida do serviço e da forma como será utilizado na aplicação.**

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/MEC - JB

.NET CORE 8

Indicações a seguir aquando da escolha de qual o método a utilizar:

- **Utilizar o *AddSingleton* quando queremos criar uma única instância de um serviço e partilhá-la com toda a aplicação.**
  - **Isto é útil para serviços que não mudam durante a vida útil da aplicação e podem ser reutilizados por diferentes componentes.**

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/AC – JB

.NET CORE 8

- **Utilize o *AddScoped* quando pretendemos criar uma instância de um serviço que é partilhado num único pedido de cliente.**
  - **Isto é útil para serviços que necessitam de manter o estado ou os dados enquanto lidam com um único pedido HTTP.**
  - **O AddScoped é o *LifeTime Service* mais utilizado em aplicações ASP .NET Core.**
    - **DbContext, repositórios e serviços relacionados com a base de dados devem ser registados utilizando o *Scoped Service***

ProgramaçãoWeb – 2024/2025 – TPLD+CE+PL – ISEC/IPC – JB

.NET CORE 8

- **Utilize o *AddTransient* quando pretendemos criar uma nova instância de um serviço sempre que solicitado.**
  - **Isto é útil para serviços que executam operações leves e sem estado.**

Programação Web – 2024/2025 – LEI-CE+PL – ISEC/ACG – JB

.NET CORE 8

Benefícios decorrentes da utilização de Injecção de Dependência

- **Capacidade de manutenção melhorada:** a base de código torna-se mais modular e mais fácil de compreender, tornando mais simples fazer alterações ou adições sem introduzir efeitos secundários indesejados.

- **Testabilidade melhorada:** as dependências podem ser facilmente simuladas ou substituídas durante os testes, permitindo testes unitários mais abrangentes e facilitando a adoção de práticas de desenvolvimento orientadas para testes.

Programação Web 2024/2025 – TPSI D+C+PL – DEC/ISEC – JB

.NET CORE 8

- **Maior escalabilidade:** A natureza fracamente acoplada da utilização deste tipo de arquitetura na aplicação permite uma extensão e integração mais fácil de novas funcionalidades ou módulos, promovendo a escalabilidade à medida que a aplicação evolui ao longo do tempo.

- **Melhor qualidade do código:** ao aderir aos princípios de separação de interesses e inversão de dependências, a qualidade geral do código é melhorada, levando a menos bugs e a processos mais fáceis de depuração e resolução de problemas.

Programação Web – 2024/2025 – LEI+CET-PL – ISEC/IPC - JB

ASP.NET CORE 8 - *SIGNALR*



- 2.11. SignalR
- É uma biblioteca integrada na framework
- Permite desenvolver funcionalidades "*real-time*" numa aplicação *ASP.NET Core*
  - Possibilita que o *server-side* faça, em tempo real, *push* aos clientes conectados
    - Ex: *Chat, Cotações da bolsa*
- Usa *WebSockets* quando disponível
- Actualmente é também usada para implementar o *Blazor WASM*

Programação Web 2024/2025 – LEI D+CE+RAS – ISEC/DEI – JPC - JB

ASP.NET CORE 8 - *SIGNALR*



- O SignalR é uma biblioteca que simplifica a adição de funcionalidades web em tempo real às aplicações.
  - Permite que o código do lado do servidor envie instantaneamente conteúdo para clientes ligados.
  - O SignalR trata da gestão de ligações automaticamente e está dimensionado para lidar com um grande número de ligações.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC

ASP.NET CORE 8 - *SIGNALR*



- **Principais características do SignalR**
  - Comunicação em tempo real: permite a comunicação bidirecional entre servidor e cliente.
  - Religação automática: volta a ligar-se automaticamente se a ligação for interrompida.
  - Múltiplas opções de transporte: escolhe o melhor método de transporte (WebSockets, eventos enviados pelo servidor, pesquisa longa) em função das capacidades do cliente.

Programação Web – 2024/2025 – LEI D+GI+PL – ISEC/IPC - JB

ASP.NET CORE 8 - *SIGNALR*

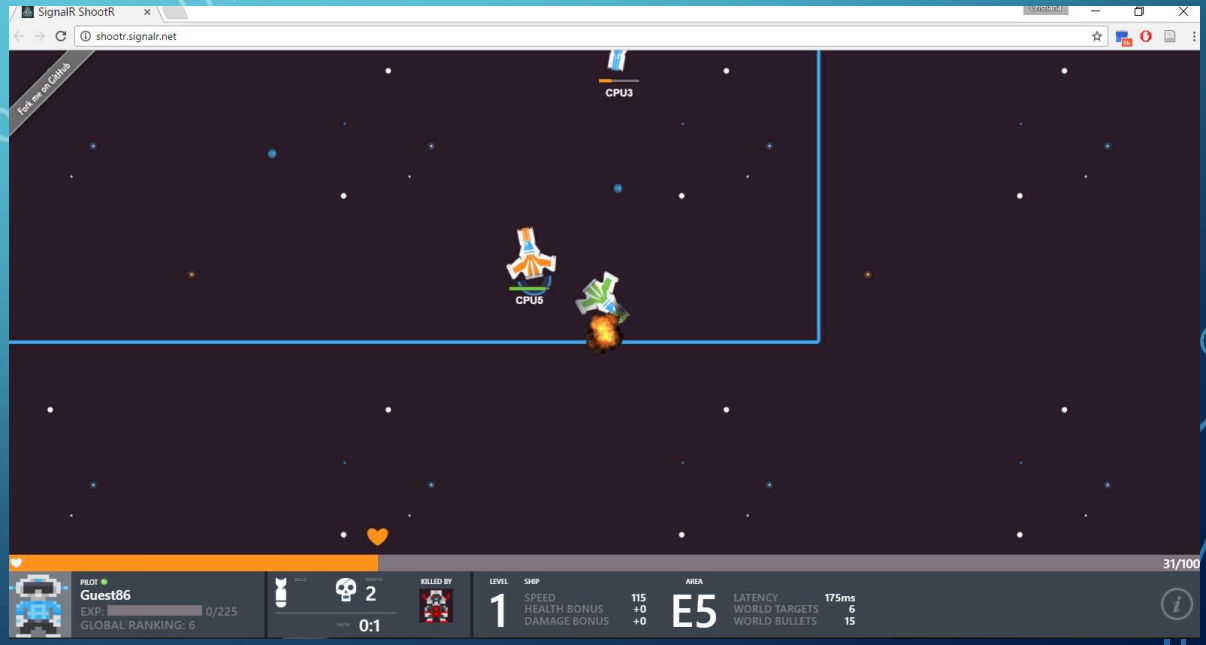


- A implementação de aplicações em tempo real com SignalR no . NET Core melhora a interatividade e a capacidade de resposta das aplicações web.
  - Permite a criação de recursos robustos em tempo real, como aplicações de chat e notificações.
  - O SignalR simplifica as complexidades da comunicação em tempo real, tornando-o uma ferramenta essencial

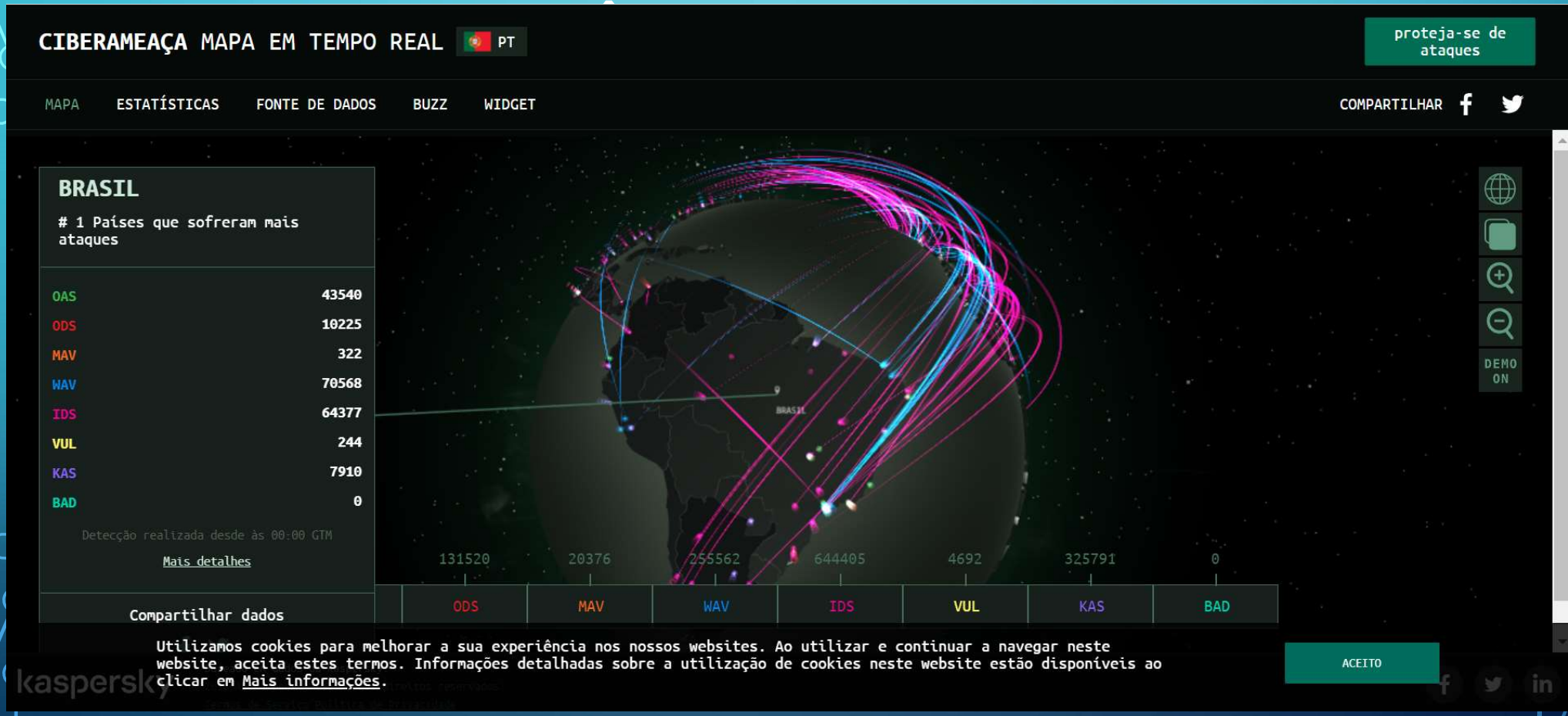
Programação Web – 2024/2025 – LEI DTSE+PL – ISEC/IPC – DS

ASP.NET CORE 8 - SIGNALR

- Permite desenvolver tipos de aplicações com alta frequência de atualizações a partir do servidor
  - Ex. Jogos em tempo real
  - Usado no Blazor



Programação Web – 2024/20 D+CE+PL – ISEC/IPC - JB



https://cybermap.kaspersky.com/pt

.NET CORE 8

2.12. Regras de Negócio

Numa aplicação empresarial complexa, a implementação eficiente de regras de negócio é crucial para manter o código limpo, evitar duplicações e garantir a escalabilidade bem como fazer com que a aplicação Web deenvolvida corresponde às necessidades e expectativas dos seus utilizadores

Programação Web – 2024/2025 – LEI+D+CET – ISEC/IPC - JB

O que são as Regras de Negócio?

As regras de negócio são condições e lógicas que definem ou restringem a forma como os dados podem ser processados dentro de um sistema.

Exemplos:

- Pedido de empréstimo: Um pedido de empréstimo só pode ser aprovado se a chamada Taxa de Esforço do cliente for inferior a determinada percentagem, por exemplo inferior a 30% ou 40%.

Nota: A taxa de esforço é o indicador que traduz o peso dos empréstimos nos rendimentos do agregado familiar.

Programação Web 2024/2025 – LEI+CE+PL – ISEC/DEI - JB

.NET CORE 8

- Lógica de desconto no comércio eletrónico: Os descontos só podem ser aplicados se o valor da compra for superior a determinado valor, por exemplo 60 €, e o cliente for um cliente que tenha subscrito um cartão de fidelização.

Escrever estas regras sem estrutura leva frequentemente a *código esparguete* – instruções if aninhadas e lógica dispersa por vários serviços.

Em .Net Core, a implementação destas regras de negócios é facilitada pela utilização de *Design Patterns*

Programação Web-T 2024/2025 – LEI L1-CE+PL – ISEC/APC - JB

.NET CORE 8

2.13. Docker no .Net Core 8

O que é o Docker?

O Docker é uma plataforma de código aberto que permite aos programadores construir, implementar, executar, atualizar e gerir aplicações em contentores de forma fácil e eficiente.

Alguns dos principais motivos da popularidade do Docker são:

- Agiliza o fluxo de trabalho de desenvolvimento,
- Reduz o problema "funciona na minha máquina",
- Permite uma implementação e dimensionamento eficiente de aplicações.

Programação Web 2024/2025 – LEI+CE-PL – ISEC/IPC - JB

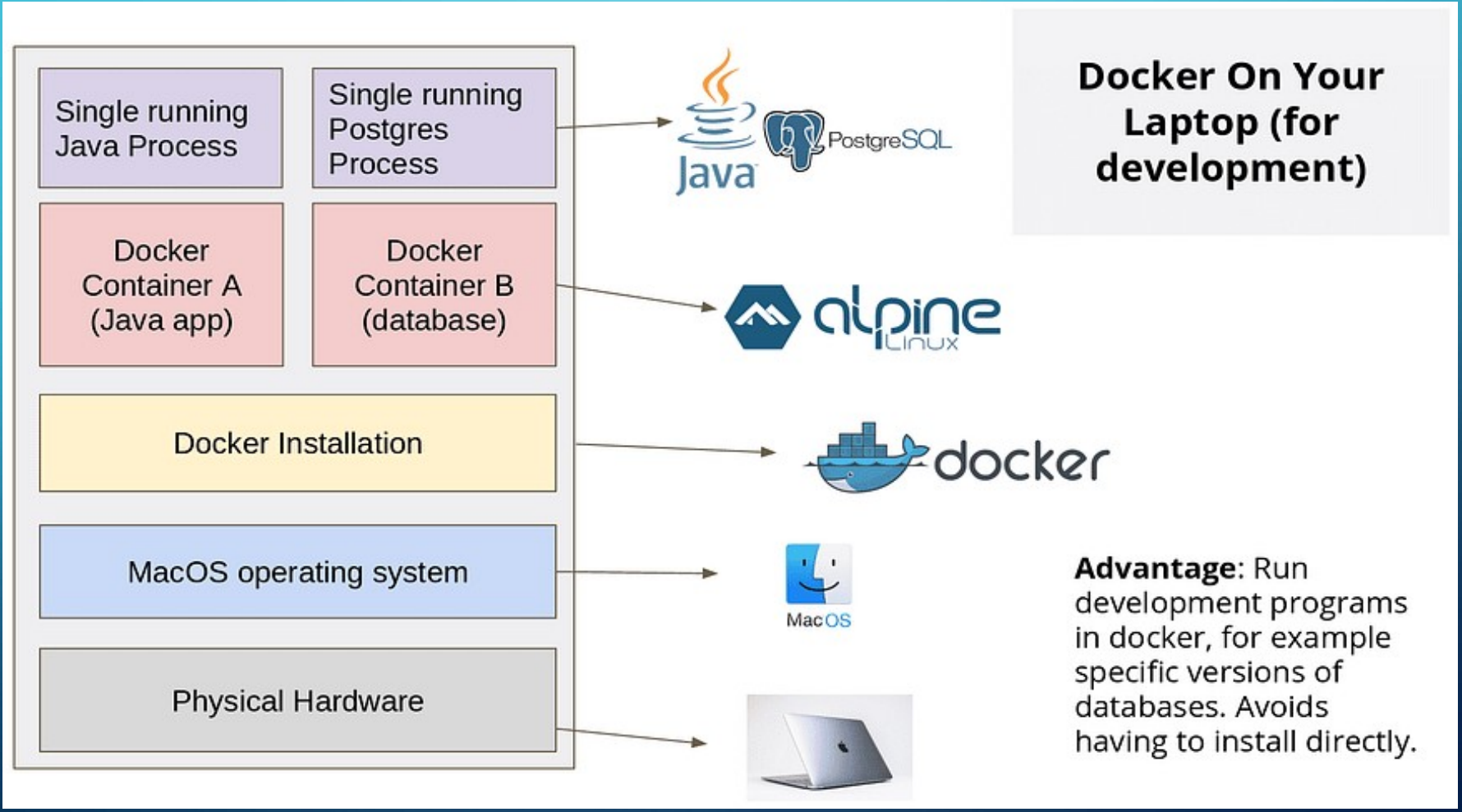
.NET CORE 8

O Docker permite aos programadores empacotar aplicações em contentores – componentes executáveis padronizados que combinam o código-fonte da aplicação com as bibliotecas do sistema operativo (SO) e as dependências necessárias para executar esse código em qualquer ambiente.

O Docker revolucionou a forma como as aplicações são desenvolvidas, distribuídas e executadas.

Programação Web – 2024/2025 – LEI+LCE+LIG – ISEC/ICT - JB

.NET CORE 8


Single running Java Process
Single running Postgres Process
Docker Container A (Java app)
Docker Container B (database)
Docker Installation
MacOS operating system
Physical Hardware
Java
PostgreSQL
alpine Linux
docker
MacOS
Docker On Your Laptop (for development)
Advantage: Run development programs in docker, for example specific versions of databases. Avoids having to install directly.

Comparação com as máquinas virtuais tradicionais

- Ao contrário das máquinas virtuais (VMs), que exigem um sistema operativo host completo para cada instância, os contentores Docker partilham o kernel do sistema operativo anfitrião e isolam as aplicações no espaço do utilizador.
  - Desse modo, os contentores Docker são muito mais leves e rápidos de iniciar em comparação com as VMs.

Programação Web – 2024/2025 – LEI+CE+PL ISEC/IPC JB

.NET CORE 8

História do Docker

O Docker foi lançado pela primeira vez em março de 2013 pela DotCloud (agora Docker, Inc.).

- O conceito original do Docker era criar uma forma leve, portátil e eficiente de empacotar e executar aplicações de forma consistente em diferentes ambientes.
- Foi inspirado no conceito dos contentores de transporte marítimo
  - Desse modo, as aplicações e as suas dependências são empacotadas em contentores padronizados que podem ser facilmente movidos e implantados.

Programação Web – 2024/2025 LEI DI CE+PL – ISEC/InC - JB

.NET CORE 8

Actualmente, o Docker é amplamente utilizado em vários setores e é apoiado por uma grande e ativa comunidade de programadores e colaboradores.

Tornou-se uma tecnologia fundamental para a contentorização e desencadeou o crescimento de outras tecnologias relacionadas, como o Kubernetes para a orquestração de contentores.

Programação Web – 2024/2025 – LEI DI-CE+PL – ISEC/IPC – JB

.NET CORE 8

Conceitos do Docker

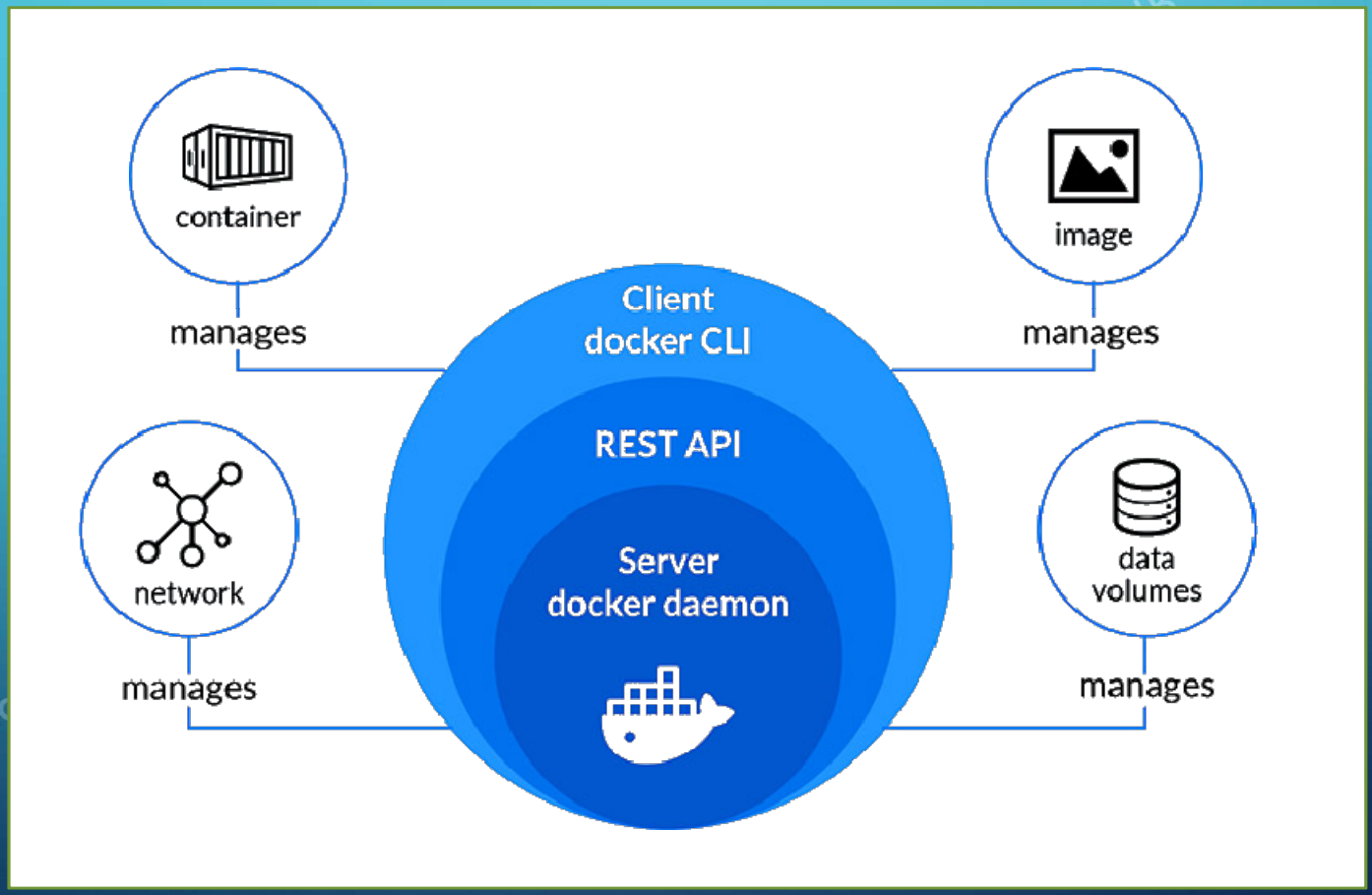
Os principais conceitos do docker a considerar são:

- Instalação,
- Comandos,
- Construção de imagens,
- Gestão de contentores,
- Rede,
- Persistência de dados,
- Outros tópicos avançados.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

Arquitectura do Docker



.NET CORE 8

Arquitectura do Docker

O Docker utiliza uma arquitetura cliente-servidor com vários componentes principais:

1. **Cliente Docker**: a ferramenta de interface de linha de comandos (CLI) que permite aos utilizadores interagir com o Docker. Comunica com o daemon Docker para executar comandos.

Programação Web - 2024/2025 – LEI D+CE+PL – ISEC/AnC - JB

.NET CORE 8

2.° **Docker Daemon (ou Docker Engine):** O Docker Engine é uma tecnologia de contentorização de código aberto que permite aos programadores empacotar aplicações em contentores.

Os contentores são componentes executáveis normalizados que combinam o código-fonte da aplicação com as bibliotecas do sistema operativo (SO) e as dependências necessárias para executar este código em qualquer ambiente. Escuta as solicitações da API Docker e processa-as de acordo.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – IB

.NET CORE 8

3. **Containered:** um componente central que gere o ciclo de vida dos contentores, incluindo iniciar, interromper e gerir processos de contentores.

4. **Runc:** Uma ferramenta CLI leve para criar e executar contentores de acordo com a especificação Open Container Initiative (OCI).

Programação Web – 2024/2025 – LEI-L+CE+PL – ISEC/IPC - JB

.NET CORE 8

5.° **Docker Registry: é** um serviço que armazena e distribui imagens Docker. O Docker Hub é o registo público padrão, mas também podem ser utilizados registos privados. É semelhante ao github, mas permite enviar imagens em vez de código fonte.

6. **Docker Networking:** Fornece recursos de rede aos contentores, permitindo que estes comuniquem entre si e com o mundo exterior.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – JB

.NET CORE 8

7. **Docker Volumes e Bind Mounts:** permitem a persistência e a partilha de dados entre contentores e o sistema anfitrião.

8. **Docker Compose:** Uma ferramenta para definir e executar aplicações com vários contentores utilizando um ficheiro YAML.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/MFC – JB

Imagens Docker

- Uma imagem Docker é um pacote executável leve e independente que inclui tudo o que é necessário para executar um software, incluindo código compilado, Runtime, bibliotecas, variáveis de ambiente e ficheiros de configuração.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

Imagens x Contentores Docker

- **Imagens Docker:** modelos imutáveis que definem o conteúdo de um contentor.

- **Docker Containers:** instâncias de tempo de execução de imagens Docker, incluindo a aplicação e as suas dependências.

Programação Web 2024/2025 – LEI D+CE+PL ISEC/IPC - JB

.NET CORE 8

2.9. Docker no .Net Core 8

Principais vantagens do Docker:

- Portabilidade: as aplicações são **executadas** da mesma forma, independentemente do **local** onde são implantadas.
- Consistência: as aplicações **comportam-se** da mesma forma, independentemente **de onde** são executadas.
- Escalabilidade: permite aumentar ou diminuir facilmente as aplicações

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

.NET CORE 8

- **Isolamento**: Os contentores encapsulam uma aplicação e as suas dependências, garantindo que são executados de forma independente uns dos outros. Cada contentor é executado no seu ambiente isolado, melhorando a segurança e a estabilidade.

- **Eficiência**: Os contentores são leves e partilham o kernel do sistema operativo host, utilizando menos recursos do sistema do que as máquinas virtuais.

Programação Web – 2024/2025 – LEI,D+CA+PL – ISEC/IPC – JB

2.14. Pontos Chave da Programação Web com C# 12 e .Net Core 8

1. Diferença entre IEnumerable<T> e ICollection<T>

Ao escrever lógica em C#, os programadores encontram frequentemente IEnumerable<T> e ICollection<T>. Embora semelhantes, servem propósitos diferentes:

- **IEnumerable<T>** permite a iteração sobre uma coleção, mas não permite a modificação dos seus elementos.
- **ICollection<T>** estende IEnumerable<T> e permite modificações, como adicionar ou remover elementos.

Programação Web – 2024/2025 – LEI-D+CE-PL – ISEC/IPC - JB

2. Injecção de Dependência no .NET Core

A **Injeção de Dependência (DI)** é um padrão de design utilizado para implementar o acoplamento fraco

No . NET Core, o DI está integrado e ajuda a gerir eficientemente as dependências entre classes.

- Registo de Serviços
- Injecção de Construtores

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

3. Diferença entre os parâmetros ref e out

Em C#, ref e out permitem a passagem de argumentos por referência, mas têm utilizações distintas:

- **ref**: O parâmetro deve ser inicializado antes de ser passado.
- **out**: O parâmetro não necessita de ser inicializado, mas o método deve atribuir um valor.

Programação Web 2024/2025 – LEI,LD+CC+PL – ISEC/IPC - JB

4. Async e Await: Melhoram o desempenho das aplicações

A programação assíncrona ajuda a aumentar o desempenho, permitindo que as operações sejam executadas sem bloquear o thread principal.

- A utilização das palavras-chave async e await permitem esse tipo de programação

Programação Web – 2024/2025 – LEI D+CF+PL – ISEC/IPC - JS

5. Tratamento de Excepções em aplicações .NET Core

O tratamento adequado das excepções é crucial para a construção de aplicações robustas

Para isso é essencial fazer-se o:

- Tratamento de excepções locais
- Tratamento Global de Excepções

Programação Web – 2024/2025 – LEI D-TGPSI/PL – ISEC/IPC - JS

6. A função de appsettings.json no ASP.NET Core

O ficheiro **appsettings. json** armazena os valores de configuração como ***Connections Strings***.

- Estas definições podem ser acedidas usando *IConfiguration*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

7.º Compreender *Task* versus *Thread* em C#

- **Thread**: Uma instrução de baixo nível para execução paralela.
- **Task**: Uma abstração de nível superior para operações assíncronas.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

8.º O que é o middleware no ASP.NET Core?

- Os componentes de middleware processam pedidos e respostas HTTP no ASP.NET Core

Programação Web – 2024/2025 – LEI DI CF+PL – ISEC/IPC - JB

9.º Diferença entre Tipos de Valor e Tipos de Referência em .NET.

- Os tipos de valor (por exemplo, int, double, struct) armazenam os dados diretamente e são alocados na pilha. Têm um tamanho fixo e são copiados quando passados para os métodos.

- Os tipos de referência (por exemplo, classe, string, objeto) armazenam referências (ponteiros) para os dados, que são alocados no heap. Quando é passada para os métodos, a referência é copiada, mas os dados subjacentes permanecem partilhados entre o método e quem o chama.

Programação Web – 2024/2025 – LEI+CET+LI – ISEC/IPC - JB

- **10. Diferença entre classe abstrata e interface**

- • ***classe abstrata***: Pode conter métodos abstratos e concretos.

- • ***interface***: Só pode definir assinaturas de métodos.

Programação Web – 2024/2025 – LEI DEISE+PL – ISEC/IPC

- **11. Genéricos em .NET**

  - Os genéricos permitem que os programadores criem estruturas ou métodos de dados com segurança de tipo

12. Métodos de extensão em .NET

- Os métodos de extensão adicionam funcionalidades aos tipos existentes sem os modificar

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

.NET CORE 8

- **14.º Pacotes NuGet em .NET**
  - O NuGet é o gestor de pacotes do .NET, disponibilizando bibliotecas e ferramentas para agilizar o desenvolvimento. Podem-se instalar pacotes através do Visual Studio ou da consola

- **15.º .NET Garbage Collector**
  - O garbage collector (GC) trata automaticamente da gestão de memória, libertando memória para os objetos que já não estão em uso. Este processo funciona em três gerações para otimizar o desempenho

Programação Web 2024/2025 – LEI+CE+RC – ISEC/IPC - JB

2.15. Novidades previstas para o C# 13 e .Net 9

1. New Lock Object

O C# 13 introduz um novo tipo System.Threading.Lock para exclusão mútua. Historicamente, um tipo de objeto era utilizado para fins de bloqueio, mas agora existe um tipo Lock dedicado,

- **Código mais limpo e seguro.** Isto torna o código mais legível e previsível. Além disso, o compilador avisa se uma instância de Lock for utilizada incorretamente como um objeto regular.

Programação Web – 2024/2025 – LEI-D+CE+TT – ISEC/IPC - JB

.NET CORE 8

- **Desempenho.** Segundo a Microsoft, pode ser mais eficiente do que bloquear uma instância de objeto arbitrária.
- **Novo Mecanismo de Bloqueio.** EnterScope substitui a classe Monitor nos bastidores. Encontre o código IL do exemplo acima aqui. Devolve uma estrutura ref que segue o padrão Dispose, permitindo uma utilização contínua com a instrução using.

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

.NET CORE 8

- **Limitações Assíncronas.** as chamadas assíncronas ainda não são permitidas nos blocos de bloqueio devido a limitações inerentes à forma como os bloqueios e o código assíncrono interagem.
  - Nestas situações pode-se utilizar a abordagem tradicional recorrendo-se aos SemaphoreSlim

Programação Web – 2024/2025 – LEI DEI-ISEC/IPC – JB

.NET CORE 8

2. Task.WhenEach

Supondo que se tem uma lista de tarefas que terminam em intervalos diferentes e se quer processar cada um assim que estiver concluído.

Em vez de se usar WaitAll() que não funcionará neste caso, pois aguarda a conclusão de todas as tarefas, pode-se usar Task.WaitAny() como solução alternativa para escolher o próximo assim que estiver concluído. O C# 13 introduz uma abordagem mais elegante e eficiente para lidar com este cenário: **Task.WhenEach.**

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/ nFC - JB

.NET CORE 8

3. *params Collections*

A partir do C# 13, os parâmetros ***params*** podem ser de qualquer tipo com suporte para expressões de coleção.

- **Código mais limpo.** Reduz significativamente o número de chamadas a . ToArray(), . ToList().
- **Desempenho.** Chamadas como . ToArray(), . ToList() já adicionam sobrecarga extra de recursos e agora suportam também a passagem de Span<> e IEnumerable<>, aproveitando a utilização de memória mais eficiente e a lazy execution..

Programação Web – 2024/2025 – LEI 3+CE+PL – ISEC/nFC - JB

.NET CORE 8

4. Propriedades semiautomáticas

Quando se declara uma propriedade implementada automaticamente em C#

```
public int Number { get; conjunto; }
```

o compilador gera um campo de suporte (por exemplo, _number) e métodos getter/setter internos void set_Number(int number) e int get_Number().

Caso seja necessário utilizar lógica personalizada, como validação, valores padrão, cálculos ou carregamento lento no getter ou setter de uma propriedade, geralmente terá de definir manualmente o campo de suporte na sua classe.

Programação Web – 2024/2025 – LEI L+CE+PL – ISEC/AIC - JB

.NET CORE 8

O C# 13 simplifica isto introduzindo a palavra-chave **field**, permitindo aceder ao campo de suporte diretamente, sem o definir manualmente.

- **Código standard reduzido.** Elimina a necessidade de definir manualmente campos de apoio privados, resultando num código mais limpo e conciso.
- **Legibilidade melhorada.** Não há necessidade de gerir nomes de campos de suporte personalizados, tornando o uso da palavra-chave field um padrão que melhora a clareza do código.

Programação Web – 2024/2025 – LEI – CE+PL – ISEC/IPC – JB

.NET CORE 8

- Property-Scoped Field. O campo de apoio privado está confinado à própria propriedade, evitando o uso não intencional noutras partes da classe. Este encapsulamento melhora a segurança e aborda uma fonte comum de erros
- Potential Breaking Change. Se a classe já tiver uma propriedade chamada field do mesmo tipo, esta terá precedência sobre a palavra-chave new field, levando potencialmente a um comportamento inesperado.

Programação Web – 2024/2025 – LEI D-CE+PL – ISEC/IE – JB

.NET CORE 8

5. Cache Híbrida

A nova API HybridCache aborda certas lacunas nas APIs IDistributedCache e IMemoryCache existentes, como o ***stampede problem***

- Introduz novas características e capacidades e torna a cache no .NET mais flexível e de alto desempenho.
- O HybridCache foi concebido para ser um substituto imediato para a maioria dos cenários IDistributedCache e IMemoryCache.

Programação Web - 2024/2025 – LEI D+CE+PL - ISEC/PCI - JB

.NET CORE 8

- O melhor dos dois mundos. O HybridCache oferece flexibilidade para armazenar dados em memória (L1) ou numa cache distribuída (L2), utilizando uma API unificada e concisa para ambos.
  - Esta configuração permite um acesso local rápido a dados utilizados com frequência (L1) e uma cache externa escalável (L2) que pode lidar com dados maiores e acedidos com menos frequência.
  - Os HybridCacheEntryFlags destinam-se a controlar este comportamento.

Programação Web – 2024/2025 – LEI DI-ESTG+PL-ISEC/IPC – JB

.NET CORE 8

- Proteção contra o ***Stampede problem***

Tanto o IMemoryCache como o IDistributedCache sofrem do ***stampede problema***

- O HybridCache resolve isto garantindo que apenas um chamador volta a gerar o valor de uma determinada chave, enquanto outros aguardam pelo resultado.

- Isto ajuda a evitar o repovoamento excessivo da cache.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/ACF - JB